

6. What are process framework activities?

A generic process framework for software engineering encompasses **five** activities

Communication

Planning

Modeling.

Construction.

Deployment.

7. Define managed level in CMMI.

- **Managed.** All level 1 criteria have been satisfied. In addition, all work associated with the process area conforms to an organizationally defined policy; all people doing the work have access to adequate resources to get the job done; stakeholders are actively involved in the process area as required; all work tasks and work products are —monitored, controlled, and reviewed;

8. Define process pattern.

- A **process pattern** describes a process-related problem that is encountered during software engineering work, identifies the environment in which the problem has been encountered, and suggests one or more proven solutions to the problem. Stated in more general terms, a process pattern provides you with a template —**a consistent method for describing problem solutions within the context of the software process.**

9. What are the phases in water fall model?

Phases in water fall model are

Communication

Planning

Modeling.

Construction.

Deployment.

10. List phases in agile model.

Following are the phases in the Agile model are as follows:

1. Requirements gathering
2. Design the requirements
3. Construction/ iteration
4. Testing/ Quality assurance
5. Deployment

1.Explain characteristics of software.**Characteristics of software**

Software has characteristics that are considerably different than those of hardware:

1) Software is developed or engineered, it is not manufactured in the Classical Sense.

Although some similarities exist between software development and hardware manufacture, the two activities are fundamentally different. In both the activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are nonexistent or easily corrected for software. Both the activities are dependent on people, but the relationship between people is totally varying. These two activities require the construction of a "**product**" but the approaches are different. Software costs are concentrated in engineering which means that software projects cannot be managed as if they were manufacturing.

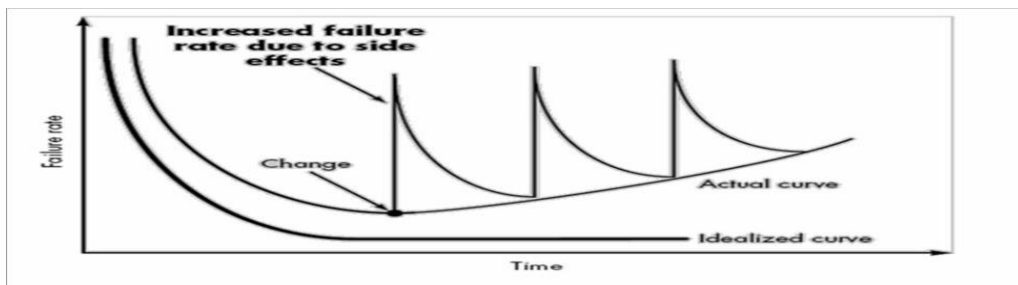


Fig: FAILURE CURVE FOR SOFTWARE

2) Software doesn't "Wear Out"

The following figure shows the relationship between failure rate and time. Consider the failure rate as a function of time for hardware. The relationship is called **the bathtub curve**, indicates that hardware exhibits relatively high failure rates early in its life, defects are corrected and the failure rate drops to a steady-state level for some period of time. As time passes, however, the failure rate rises again as hardware components suffer from the cumulative

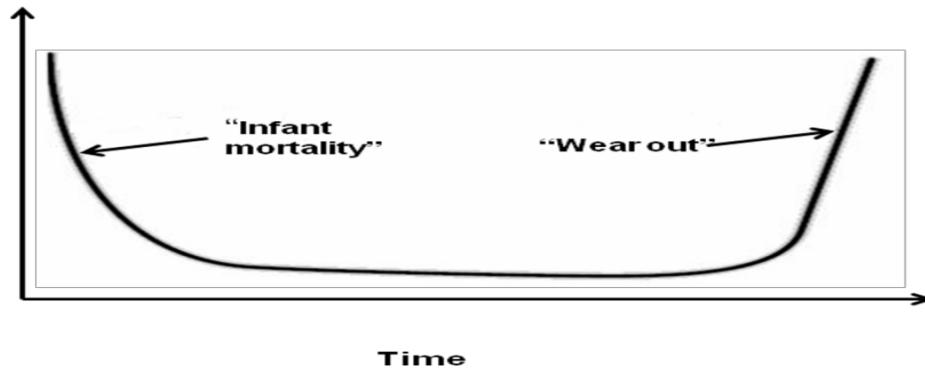


Fig: FAILURE CURVE FOR HARDWARE

effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. So, stated simply, the hardware begins to wear out. Software is not susceptible to the environmental maladies that cause **hardware to wear out**

3) Although the industry is moving toward component-based construction, most software continues to be custom built

A software component should be designed and implemented so that it can be reused in many different programs. Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the software engineer to create new applications from reusable parts

2.Explain about the changing nature of software

THE CHANGING NATURE OF SOFTWARE:

The 7 broad categories of computer software present continuing challenges for software engineers:

- 1) System software
- 2) Application software
- 3) Engineering/scientific software
- 4) Embedded software
- 5) Product-line software
- 6) Web-applications
- 7) Artificial intelligence software.

System software : A collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities)

- The systems software is characterized by

- - heavy interaction with computer hardware
- - heavy usage by multiple users
- **Application software** : Stand-alone programs that solve a specific business need. Application software is used to control business functions in real time (e.g., point-of-sale transaction processing, real-time manufacturing process control).
- **Engineering/scientific software** : It has been characterized by “number crunching” algorithms. Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing.
- **Embedded software** : It resides within a product or system and is used to implement and control features and functions for the end user and for the system itself. Embedded software can perform limited and esoteric functions (e.g., key pad control for a microwave oven) or provide significant function and control capability (e.g., digital functions in an automobile such as fuel control)
- **Product-line software** : Designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products) or address mass consumer markets (e.g., word processing, spreadsheets, computer graphics, multimedia, entertainment, database management, and personal and business financial applications).
- **Web applications** : These Applications called “WebApps,” this network-centric software category spans a wide array of applications. In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics.
- **Artificial intelligence software** : These makes use of non numerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

3. Discuss about software myths.

Software Myths

Software Myths- beliefs about software and the process used to build it - can be traced to the earliest days of computing. Myths have a number of attributes that have made them insidious.

Management Myths :

Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality.

Myth : *We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?*

- **Reality** :
 - The book of standards may very well exist, but is it used?
 - Are software practitioners aware of its existence?
 - Does it reflect modern software engineering practice?
 - Is it complete?
 - Is it adaptable?
 - Is it streamlined to improve time to delivery while still maintaining a focus on Quality? In many cases, the answer to these entire question is NO.
- **Myth** : *If we get behind schedule, we can add more programmers and catch up*
- **Reality** : Software development is not a mechanistic process like manufacturing. “Adding people to a late software project makes it later.” At first, this statement may seem counterintuitive. However, as new people are added,

people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort

- **Myth :** *If we decide to outsource the software project to a third party, I can just relax and let that firm build it.*
- **Reality :** If an organization does not understand how to manage and control software project internally, it will invariably struggle when it out sources software project.

Customer Myths:

A customer who requests computer software may be a person at the next desk, a technical group down the hall, the marketing /sales department, or an outside company that has requested software under contract.

Myth : *A general statement of objectives is sufficient to begin writing programs - we can fill in details later.*

- **Reality :** Although a comprehensive and stable statement of requirements is not always possible,
- an ambiguous statement of objectives is a recipe for disaster. Unambiguous requirements are developed only through effective and continuous communication between customer and developer.
- **Myth :** *Project requirements continually change, but change can be easily accommodated because software is flexible.*
- **Reality :** It's true that software requirement change, but the impact of change varies with the time at which it is introduced. When requirement changes are requested early, cost impact is relatively small. However, as time passes, cost impact grows rapidly – resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification

Practitioner's myths.

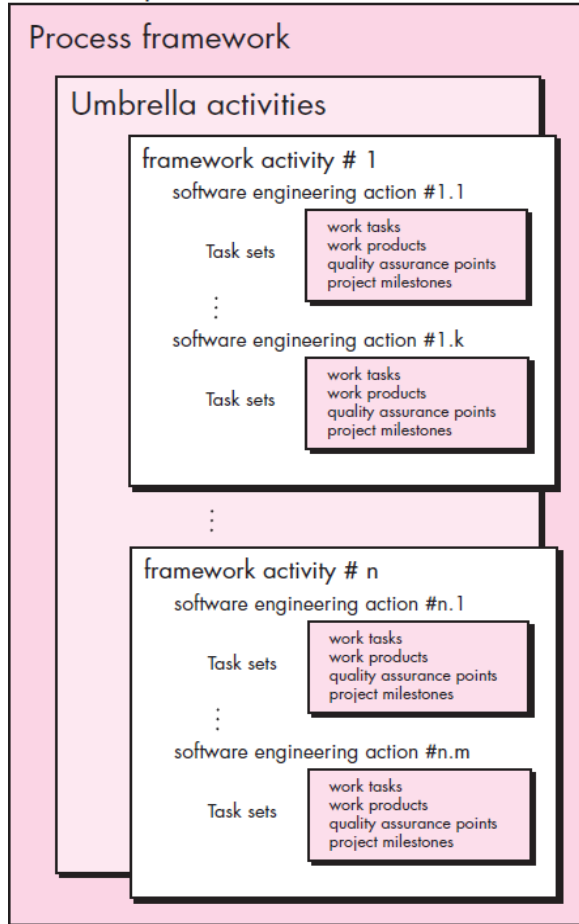
Myths that are still believed by software practitioners have been fostered by 50 years of programming culture. During the early days of software, programming was viewed as an art form. Old ways and attitudes die hard.

- **Myth:** Once we write the program and get it to work, our job is done.
- **Reality:** Someone once said that "the sooner you begin 'writing code', the longer it'll take you to get done." Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.
- **Myth:** Until I get the program "running" I have no way of assessing its quality.
- **Reality:** One of the most effective software quality assurance mechanisms can be applied from the inception of a project—the formal technical review. Software reviews are a "quality filter" that have been found to be more effective than testing for finding certain classes of software defects.
- **Myth:** The only deliverable work product for a successful project is the working program.
- **Reality:** A working program is only one part of a software configuration that includes many elements. Documentation provides a foundation for successful engineering and, more important, guidance for software support.
- **Myth:** Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.
- **Reality:** Software engineering is not about creating documents. It is about creating quality. Better quality leads to reduced rework. And reduced rework results in faster delivery times. Many software professionals recognize the fallacy of the myths just described.

4.Elaborate process framework.

A **process framework** establishes the foundation for a complete software engineering process by identifying a small number of **framework activities** that are applicable to all software projects, regardless of their size or complexity. In addition, the process framework encompasses a set of **umbrella activities** that are applicable across the entire software process.

Software process



A generic process framework for software engineering encompasses **five** activities:

- **Communication.** Before any technical work can commence, it is critically important to communicate and collaborate with the customer. The intent is to understand stakeholders objectives for the project and to gather requirements that help define software features and functions.
- **Planning.** A software project is a complicated journey, and the planning activity creates a “map” that helps guide the team as it makes the journey. The map—called a *software project plan*—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.
- **Modeling.** Creation of models to help developers and customers understand the requires and software design
- **Construction.** This activity combines code generation and the testing that is required to uncover errors in the code.
- **Deployment.** The software is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

These **five** generic framework activities can be used during the development of small, simple programs, the creation of large Web applications, and for the engineering of large, complex computer-based systems.

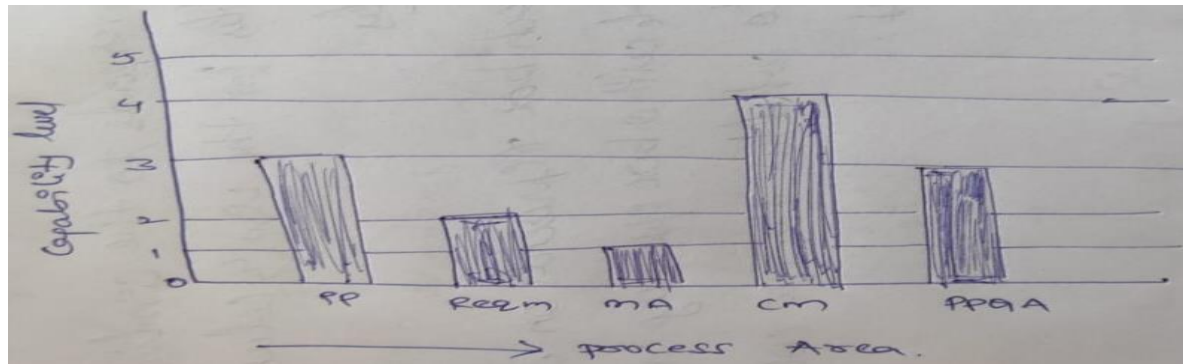
In general, **umbrella activities** are applied throughout a software project and help a software team manage and control progress, quality, change, and risk. Typical umbrella activities include:

- **Software project tracking and control**—allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.
- **Risk management**—assesses risks that may affect the outcome of the project or the quality of the product.
- **Software quality assurance**—defines and conducts the activities required to ensure software quality.
- **Technical reviews**—assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.
- **Measurement**—defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders needs; can be used in conjunction with all other framework and umbrella activities.
- **Software configuration management**—manages the effects of change throughout the software process.
- **Reusability management**—defines criteria for work product reuse and establishes mechanisms to achieve reusable components.
- **Work product preparation and production**—encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

5.Explain levels in Capability Maturity Model Integration model in continuous model with neat diagram.

The CMMI represents a process meta-model in two different ways: As a continuous model
As a staged model. Each process area is formally assessed against specific goals and practices and is rated according to the following capability levels.

- **Level 0: Incomplete.** The process area is either not performed or does not achieve all goals and objectives defined by CMMI for level 1 capability.
- **Level 1: Performed.** All of the specific goals of the process area have been satisfied. Work tasks required to produce defined work products are being conducted.
- **Level 2: Managed.** All level 1 criteria have been satisfied. In addition, all work associated with the process area conforms to an organizationally defined policy; all people doing the work have access to adequate resources to get the job done; stakeholders are actively involved in the process area as required; all work tasks and work products are —monitored, controlled, and reviewed;
- **Level 3: Defined.** All level 2 criteria have been achieved. In addition, the process is —tailored from the organizations set of standard processes according to the organizations tailoring guidelines, and contributes and work products, measures and other process-improvement information to the organizational process assets.



- **Level 4: Quantitatively managed.** All level 3 criteria have been achieved. In addition, the process area is controlled and improved using measurement and quantitative assessment. Quantitative objectives for quality and process performance are established and used as criteria in managing the process.
- **Level 5: Optimized.** All level 4 criteria have been achieved. In addition, the process area is adapted and optimized using quantitative means to meet changing customer needs and to continually improve the efficacy of the process area under consideration

The CMMI defines each process area in terms of —specific goals|| and the —specific practices|| required to achieve these goals. Specific practices refine a goal into a set of process-related activities.

6.Explain levels in capability maturity model integration in staged model.

Staged model

- This model is used if you have no clue of how to improve the process for quality software.
- Levels are called maturity levels

LEVEL	FOCUS	PROCESS AREA
Optimizing	Continuous process Improvement	-Organizational Innovation and Deployment -Causal Analysis and Resolution
Quantitatively managed	Quantitative management	-Organizational process performance -Quantitative project management

Defined	Process standardized	Requirements Development Technical Solution Product Integration Verification Validation Organizational Process Focus Organizational Training Integrated Project Management Risk Management
LEVEL	FOCUS	PROCESS AREA
Optimizing	Continuous process Improvement	-Organizational Innovation and Deployment -Causal Analysis and Resolution

Quantitatively managed	Quantitative management	-Organizational process performance -Quantitative project management
Defined	Process standardized	Requirements Development Technical Solution Product Integration Verification Validation Organizational Process Focus Organizational Training Integrated Project Management Risk Management

7.Discuss about software engineering-a layered technology.

Software Engineering - A Layered Technology

In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities

- **Problem should be understood before software solution is developed**
- **Design is a pivotal Software Engineering activity**
- **Software should exhibit high quality**

- **Software should be maintainable**

These simple realities lead to one conclusion. Software in all of its forms and across all of its application domains should be **engineered**.

Software Engineering is a **layered technology**. Software Engineering encompasses a **Process, Methods** for managing and engineering software and **tools**.

The following Figure represents **Software engineering Layers**



Software engineering is a layered technology. Referring to above Figure, any engineering approach must rest on an organizational commitment to **quality**.

The bedrock that supports software engineering is a **quality focus**.

The foundation for software engineering is the **process layer**. The software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software. **Process** defines a **framework** that must be established for effective delivery of software engineering technology.

Software engineering **methods** provide the technical **how-to's** for building software. **Methods** encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support.

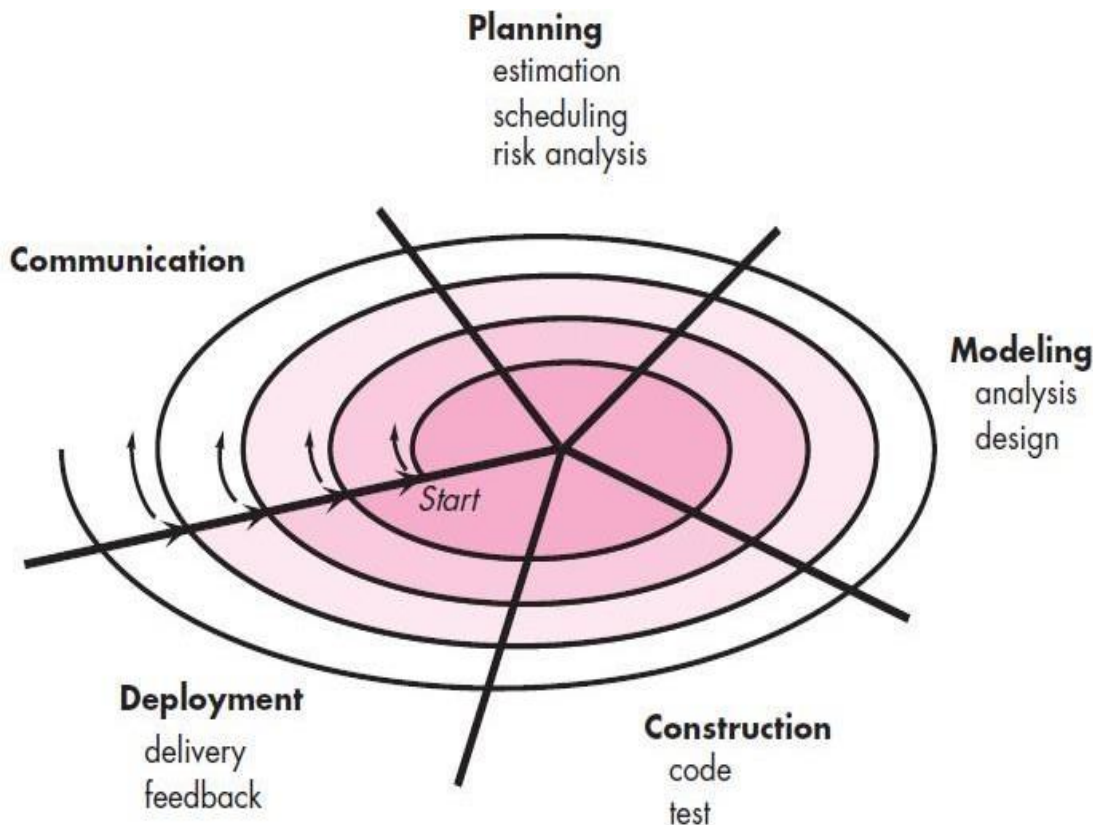
Software engineering **tools** provide **automated or semi automated** support for the process and the methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called *computer-aided software engineering*, is established.

8.Explain spiral model with diagram.

- **The Spiral Model** : Originally proposed by **Barry Boehm**, the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. It provides the potential for rapid development of increasingly more complete versions of the software. Boehm describes the model in the following manner

- The spiral development model is a **risk-driven process model** generator that is used to **guide multi-stakeholder concurrent engineering** of software intensive systems. It has **two** main distinguishing features. One is a **cyclic approach** for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk. The other is a set of **anchor point milestones** for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions.

Using the spiral model, software is developed in a series of evolutionary releases.



- A spiral model is divided into a set of **framework activities** defined by the software engineering team. As this evolutionary process begins, the software team performs activities that are implied by a circuit around the spiral in a **clockwise** direction, beginning at the **center**. Risk is considered as each revolution is made. **Anchor point milestones** are a combination of work products and conditions that are attained along the path of the spiral are noted for each evolutionary pass.
- The first circuit around the spiral might result in the development of a **product** specification; subsequent passes around the spiral might be used to develop a **prototype** and then progressively more sophisticated versions of the software. Each pass through the planning region results in adjustments to the project plan.
- The spiral model can be adapted to apply throughout the life of the computer software. Therefore, the first circuit around the spiral might represent a "**concept development project**" that starts at the core of the spiral and continues for multiple iterations until concept development is complete.
- The spiral model is a **realistic approach** to the development of **large-scale systems** and software. Because software evolves as the process progresses, the developer and customer better understand and react to risks at each evolutionary level. It maintains the systematic stepwise approach suggested by the classic life cycle but incorporates it into an iterative framework that more realistically reflects the real world

Advantages of the Spiral Model

- Risk Handling:
- Good for large projects
- Flexibility in Requirements
- Iterative and Incremental Approach
- Improved Communication
- Improved Quality

Disadvantages of the Spiral Model

- Complex
- Expensive
- Difficulty in time management
- Time-Consuming

9.Explain water fall model with neat diagram

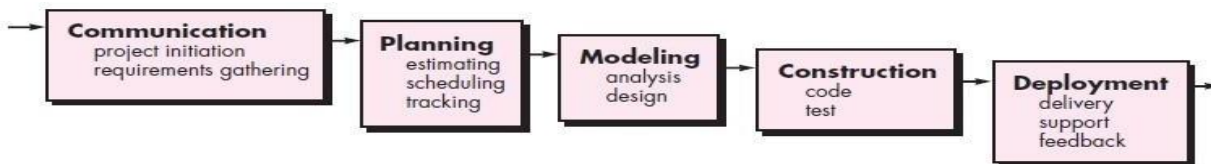
THE WATERFALL MODEL:

The waterfall model, sometimes called the *classic life cycle*, suggests a systematic sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment.

Context: Used when requirements are reasonably well understood.

Advantage:

It can serve as a useful process model in situations where requirements are fixed and work is to proceed to complete in a linear manner.



The **problems** that are sometimes encountered when the waterfall model is applied are:

Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds. The customer must have patience. A working version of the programs will not be available until late in the project time-span. If a major blunder is undetected then it can be disastrous until the program is reviewed.

Phases of SDLC Waterfall Model

1. Communication

Communication involves gathering requirements from customers and stakeholders to determine the system's objectives and the software's requirements.

2. Planning

Definition: Planning involves establishing an engineering work plan, describing technical risks, listing resource requirements, and defining a work schedule.

Activities:

- **Work Plan**
- **Risk Assessment**
- **Resource Allocation**
- **Schedule Definition:**

3. Modeling

Modelling involves creating architectural models and designs to better understand the problem and work towards the best solution.

Activities:

- **Analysis of Requirements:.**
- **Design:**

4. Construction

Construction involves creating code, testing the system, fixing bugs, and confirming that all criteria are met.

Activities:

- **Code Generation:** Writing the actual code based on the design models.
- **Testing:** Running tests to ensure the software works as intended, identifying and fixing bugs.

5. Deployment

Deployment involves presenting the completed or partially completed product to customers for evaluation and feedback, then making necessary modifications based on their input.

Activities:

- **Product Release**
- **Feedback Collection**
- **Product Improvement**

➤ **Advantages of the SDLC Waterfall Model**

Easy to Understand: The Classical Waterfall Model is very simple and easy to understand.

Individual Processing: Phases in the Classical Waterfall model are processed one at a time.

Properly Defined: In the classical waterfall model, each stage in the model is clearly defined.

Clear Milestones: The classical Waterfall model has very clear and well-understood milestones.

Properly Documented: Processes, actions, and results are very well documented.

Disadvantages of the SDLC Waterfall Model

- No Feedback Path
- Difficult to accommodate Change Requests
- Limited Flexibility:
- Limited Stakeholder Involvement:
- Late Defect Detection:

Features of the SDLC Waterfall Model

- Sequential Approach
- Document-Driven
- Quality Control
- Rigorous Planning

10.Explain about Agile methodology.

Agile Methodology

The meaning of Agile is swift or versatile. “**Agile process model**” refers to a software development approach based on iterative development. Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning. The project scope and requirements are laid down at the beginning of the development process. Plans regarding the number of iterations, the duration and the scope of each iteration are clearly defined in advance.

Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks. The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements. Each iteration involves a team working through a full software development life cycle including planning, requirements analysis, design, coding, and testing before a working product is demonstrated to the client.

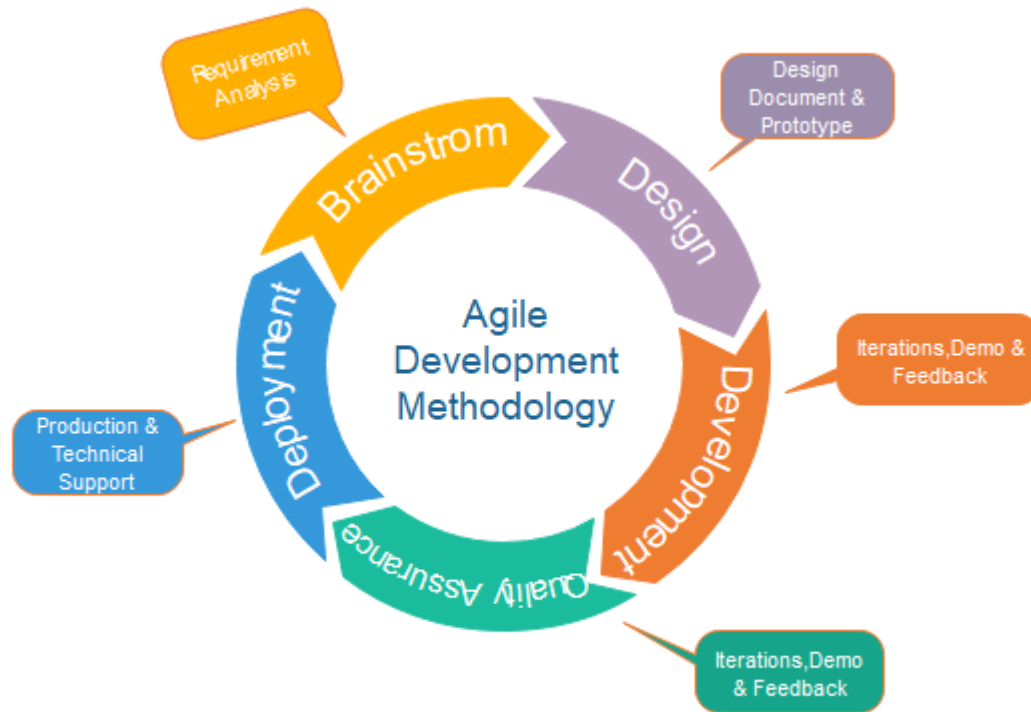


Fig. Agile Model

Phases of Agile Model:

Following are the phases in the Agile model are as follows:

7. Requirements gathering
8. Design the requirements
9. Construction/ iteration
10. Testing/ Quality assurance
11. Deployment
12. Feedback

1. Requirements gathering: In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility.

2. Design the requirements: When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.

3. Construction/ iteration: When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.

4. Testing: In this phase, the Quality Assurance team examines the product's performance and looks for the bug.

5. Deployment: In this phase, the team issues a product for the user's work environment.

6. Feedback: After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Agile Testing Methods:

- Scrum
- Crystal
- Dynamic Software Development Method(DSDM)
- Feature Driven Development(FDD)
- Lean Software Development
- eXtreme Programming(XP)

Scrum

SCRUM is an agile development process focused primarily on ways to manage tasks in team-based development conditions.

There are three roles in it, and their responsibilities are:

- **Scrum Master:** The scrum can set up the master team, arrange the meeting and remove obstacles for the process
- **Product owner:** The product owner makes the product backlog, prioritizes the delay and is responsible for the distribution of functionality on each repetition.
- **Scrum Team:** The team manages its work and organizes the work to complete the sprint or cycle.

eXtreme Programming(XP)

This type of methodology is used when customers are constantly changing demands or requirements, or when they are not sure about the system's performance.

Crystal:

There are three concepts of this method-

1. Chartering: Multi activities are involved in this phase such as making a development team, performing feasibility analysis, developing plans, etc.
2. Cyclic delivery: under this, two more cycles consist, these are:
 - A. Team updates the release plan.

B. Integrated product delivers to the users.

3. Wrap up: According to the user environment, this phase performs deployment, post-deployment.

Dynamic Software Development Method(DSDM):

DSDM is a rapid application development strategy for software development and gives an agile project distribution structure. The essential features of DSDM are that users must be actively connected, and teams have been given the right to make decisions.

The DSDM project contains seven stages:

1. Pre-project
2. Feasibility Study
3. Business Study
4. Functional Model Iteration
5. Design and build Iteration
6. Implementation
7. Post-project

Feature Driven Development(FDD):

This method focuses on "Designing and Building" features. In contrast to other smart methods, FDD describes the small steps of the work that should be obtained separately per function.

Lean Software Development:

Lean software development methodology follows the principle "just in time production." The lean method indicates the increasing speed of software development and reducing costs. Lean development can be summarized in seven phases.

1. Eliminating Waste
2. Amplifying learning
3. Defer commitment (deciding as late as possible)
4. Early delivery
5. Empowering the team
6. Building Integrity
7. Optimize the whole

When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

Advantage (Pros) of Agile Method:

1. Frequent Delivery
2. Face-to-Face Communication with clients.
3. Efficient design and fulfils the business requirement.
4. Anytime changes are acceptable.
5. It reduces total development time.

Disadvantages (Cons) of Agile Model:

1. Due to the shortage of formal documents, it creates confusion and crucial decisions taken throughout various phases can be misinterpreted at any time by different team members.
2. Due to the lack of proper documentation, once the project completes and the developers allotted to another project, maintenance of the finished project can become a difficulty.

SE UNIT-2 SOLUTIONS

SHORT ANSWER SOLUTIONS

1. Define user requirements.

User requirements

Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.

2. Define system requirements.

System requirements

A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

3. Define procedural interfaces.

Procedural interfaces where existing programs or sub-systems offer a range of services that are accessed by calling interface procedures. These interfaces are sometimes called Application Programming Interfaces (APIs)

4. What is feasibility study?

A feasibility study decides whether or not the proposed system is worthwhile. The input to the feasibility study is a set of preliminary business requirements, an outline description of the system and how the system is intended to support business processes.

5. Define requirements prioritization and negotiation

Requirements prioritization is the process of assembling requirements in ascending order of relative urgency and importance. Project constraints such as budget and time dictate the need for requirements prioritization.

Requirements negotiation is performed to address conflicts over business requirements to achieve a shared vision of software to be developed.

6.what is interactor view point give one example?

Interactor viewpoints

People or other systems that interact directly with the system. These viewpoints provide detailed system requirements covering the system features and interfaces. In an ATM, the customer's and the account database are interactor VPs.

7. what is indirect view point give one example?

Indirect viewpoints

Stakeholders who do not use the system themselves but who influence the requirements. These viewpoints are more likely to provide higher-level organisation requirements and constraints. In an ATM, management and security staff are indirect viewpoints.

8.List types of interviewing.

There are two types of interviews

- **Closed** interviews where a pre-defined set of questions are answered.
- **Open** interviews where there is no pre-defined agenda and a range of issues are explored with stakeholders.

9.what are enduring and volatile requirements?

- *Enduring requirements: Stable requirements derived from the core activity of the customer organisation.*

E.g. a hospital will always have doctors, nurses, etc. May be derived from domain models

- *Volatile requirements: Requirements which change during development or when the system is in use. In a hospital, requirements derived from health-care policy*

10. What are review checks?

Verifiability

Comprehensibility:

Traceability

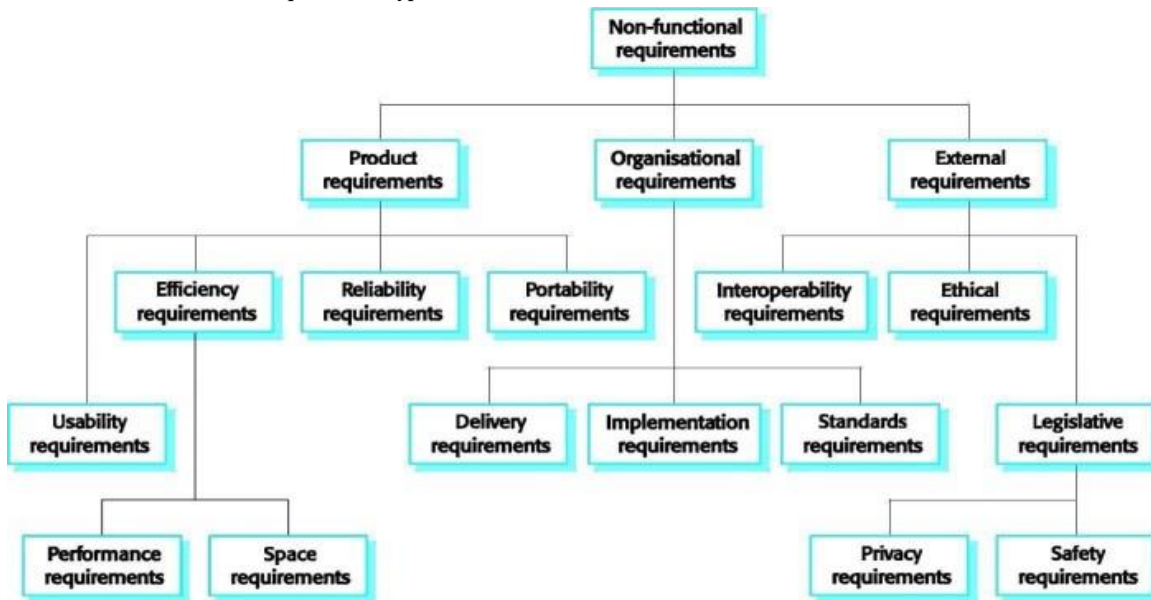
Adaptability

SE UNIT-2 LONG ANSWER SOLUTIONS

1. Discuss briefly about non-functional requirements.

NON-FUNCTIONAL REQUIREMENTS

- These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system is useless.
- Non-functional requirement types:



Non-functional requirements :

Product requirements

Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

- *Eg:* The user interface for LIBSYS shall be implemented as simple HTML without frames or Java applets.

Organisational requirements

- Requirements which are a consequence of organisational policies and procedures
- e.g. process standards used, implementation requirements, etc.

External requirements

- Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.
- *Eg:* The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system.

2. Explain about system requirements.

SYSTEM REQUIREMENTS

- More detailed specifications of system functions, services and constraints than user requirements.
- They are intended to be a basis for designing the system. They may be incorporated into the system contract.

- System requirements may be defined or illustrated using system models

Requirements and design

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, requirements and design are inseparable
- A system architecture may be designed to structure the requirements;
- The system may inter-operate with other systems that generate design requirements; The use of a specific design may be a domain requirement.

Problems with NL(natural language) specification

Ambiguity

The readers and writers of the requirement must interpret the same words in the same way. NL is naturally ambiguous so this is very difficult.

Over-flexibility

The same thing may be said in a number of different ways in the specification.

Lack of modularisation

NL structures are inadequate to structure system requirements.

Alternatives to NL specification:

Notation	Description
Structured natural language	This approach depends on defining standard forms or templates to express the requirements specification.
Design description languages	This approach uses a language like a programming language but with more abstract features to specify the requirements by defining an operational model of the system. This approach is not now widely used although it can be useful for interface specifications.
Graphical notations	A graphical language, supplemented by text annotations is used to define the functional requirements for the system. An early example of such a graphical language was SADT. Now, use-case descriptions and sequence diagrams are commonly used.
Mathematical specifications	These are notations based on mathematical concepts such as finite-state machines or sets. These unambiguous specifications reduce the arguments between customer and contractor about system functionality. However, most customers <u>don't</u> understand formal specifications and are reluctant to accept it as a system contract.

3.Explain about Software requirements documentation

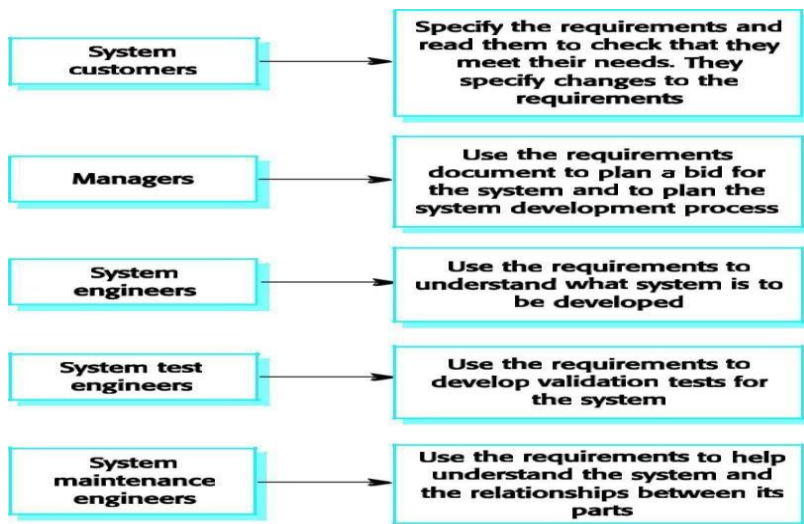
THE SOFTWARE REQUIREMENTS DOCUMENT:

The requirements document is the official statement of what is required of the system developers.

Should include both a definition of user requirements and a specification of the system requirements.

It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it

Users of a requirements document:



- **IEEE requirements standard** defines a generic structure for a requirements document that must be instantiated for each specific system.

Introduction.

- Purpose of the requirements document
- Scope of the project
- Definitions, acronyms and abbreviations
- References
- Overview of the remainder of the document

General description.

- Product perspective
- Product functions
- User characteristics
- General constraints
- Assumptions and dependencies

Specific requirements cover functional, non-functional and interface requirements. The requirements may document external interfaces, describe system functionality and performance, specify logical database requirements, design constraints, emergent system properties and quality characteristics.

- **Appendices.**
- **Index.**

4. Discuss about Feasibility study.

- A feasibility study decides whether or not the proposed system is worthwhile. The input to the feasibility study is a set of preliminary business requirements, an outline description of the system and how the system is intended to support business processes. The results of the feasibility study should be a report that recommends whether or not it worth carrying on with the requirements engineering and system development process.

A short focused study that checks

- – Does the system contributes to organisational objectives?;
- – can the system can be engineered using current technology and within budget and schedule?
- -Can the system can be integrated with other systems that are used?.
- **Feasibility study implementation:**
- A feasibility study involves information assessment, information collection and report writing.

Questions for people in the organisation

- – What if the system wasn't implemented?
- – What are current process problems?
- – How will the proposed system help?
- – What will be the integration problems?
- – Is new technology needed? What skills?
- – What facilities must be supported by the proposed system?
- In a feasibility study, you may consult information sources such as the managers of the departments where the system will be used, software engineers who are familiar with the type of system that is proposed, technology experts and end-users of the system. They should try to complete a feasibility study in two or three weeks.
- Once you have the information, you write the feasibility study report. You should make a recommendation about whether or not the system development should continue. In the report, you may propose changes to the scope, budget and schedule of the system and suggest further high-level requirements for the system.

5.Explain about requirements validation

REQUIREMENTS VALIDATION

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
- – Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking:

- *Validity: Does the system provide the functions which best support the customer's needs?*
- *Consistency: Are there any requirements conflicts?*
- *Completeness: Are all functions required by the customer included?*
- *Realism: Can the requirements be implemented given available budget and technology*
- *Verifiability: Can the requirements be checked?*

Requirements validation techniques

Requirements reviews

– Systematic manual analysis of the requirements.

Prototyping

– Using an executable model of the system to check requirements.

Test-case generation

– Developing tests for requirements to check testability.

Requirements reviews:

- Regular reviews should be held while the requirements definition is being formulated. Both client and contractor staff should be involved in reviews.
- Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks:

- *Verifiability: Is the requirement realistically testable?*
- *Comprehensibility: Is the requirement properly understood?*
- *Traceability: Is the origin of the requirement clearly stated?*
- *Adaptability: Can the requirement be changed without a large impact on other requirements?*

6.Explain about requirements management

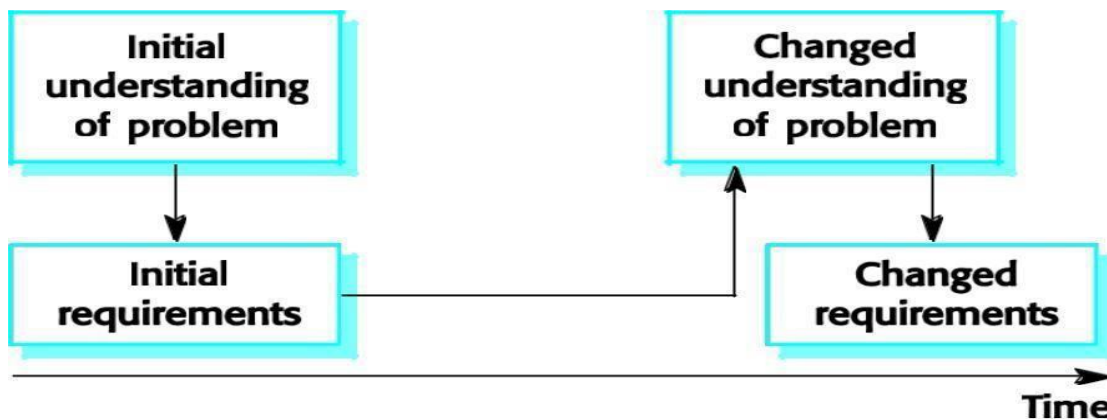
REQUIREMENTS MANAGEMENT

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- Requirements are inevitably incomplete and inconsistent
- – New requirements emerge during the process as business needs change and a better understanding of the system is developed;
- – Different viewpoints have different requirements and these are often contradictory.

Requirements change

- The priority of requirements from different viewpoints changes during the development process. System customers may specify requirements from a business perspective that conflict with end- user requirements.
- The business and technical environment of the system changes during its development.

Requirements evolution:



Enduring and volatile requirements:

- *Enduring requirements: Stable requirements derived from the core activity of the customer organisation.*

E.g. a hospital will always have doctors, nurses, etc. May be derived from domain models

- *Volatile requirements: Requirements which change during development or when the system is in use. In a hospital, requirements derived from health-care policy*

Requirements management planning:

- During the requirements engineering process, you have to plan:
- Requirements identification
- A change management process
- Traceability policies
- CASE tool support
- **Requirements change management:**

Should apply to all proposed changes to the requirements.

Principal stages

- – Problem analysis. Discuss requirements problem and propose change;
- – Change analysis and costing. Assess effects of change on other requirements;
- – Change implementation. Modify requirements document and other documents to reflect change.

Change management:



7.Explain about user requirements.

USER REQUIREMENTS

Should describe functional and non-functional requirements in such a way that they are understandable by system users who don't have detailed technical knowledge.

User requirements are defined using natural language, tables and diagrams as these can be understood by all users.

Problems with natural language

Lack of clarity

Precision is difficult without making the document difficult to read. Requirements confusion

Functional and non-functional requirements tend to be mixed-up. Requirements amalgamation

Several different requirements may be expressed together.

Requirement problems

Database requirements includes both conceptual and detailed information

- Describes the concept of a financial accounting system that is to be included in LIBSYS;

However, it also includes the detail that managers can configure this system - this is unnecessary at this level.

Grid requirement mixes three different kinds of requirement Conceptual functional requirement (the need for a grid); Non-functional requirement (grid units); Non-functional UI requirement (grid switching). Structured presentation

Guidelines for writing requirements

Invent a standard format and use it for all requirements.

Use language in a consistent way.

Use shall for mandatory requirements, should for desirable requirements.

Use text highlighting to identify key parts of the requirement. Avoid the use of computer jargon.

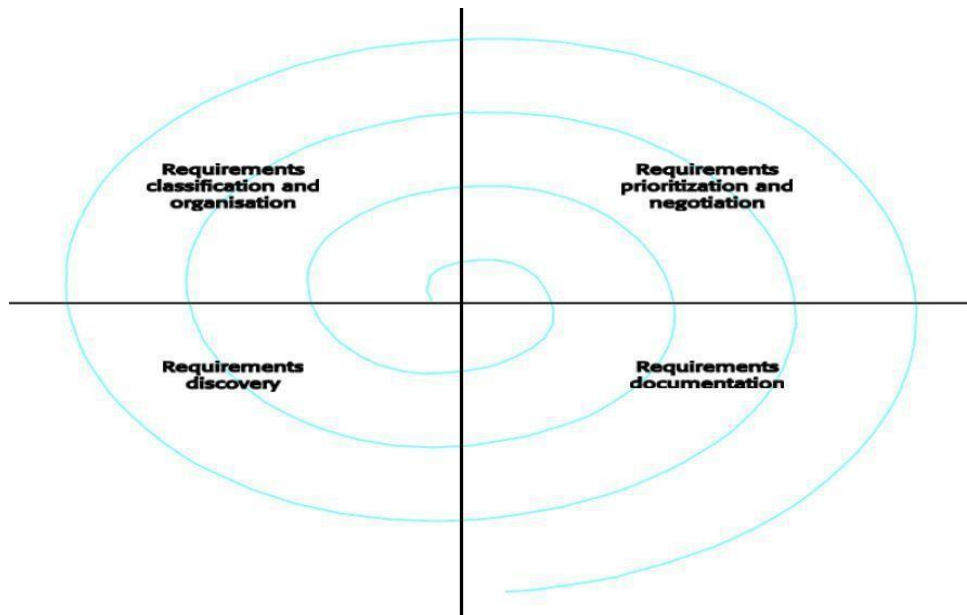
8. Discuss about requirements elicitation and analysis

REQUIREMENT ELICITATION AND ANALYSIS:

- The requirement engineering process is requirements elicitation and analysis. Sometimes called requirements elicitation or requirements discovery.
- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called *stakeholders*.

Problems of requirements analysis

- Stakeholders don't know what they really want. Stakeholders express requirements in their own terms. Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment change.



Process activities

- Requirements discovery
 - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- Requirements classification and organisation
 - Groups related requirements and organises them into coherent clusters.
- Prioritisation and negotiation
 - Prioritising requirements and resolving requirements conflicts.
- Requirements documentation
 - Requirements are documented and input into the next round of the spiral.

The process cycle starts with requirements discovery and ends with requirements documentation. The analyst's understanding of the requirements improves with each round of the cycle.

9.Explain the process of requirements discovery.

REQUIREMENTS DISCOVERY:

- Requirement discovery is the process of gathering information about the proposed and existing systems and distilling the user and system requirements from this information.
- Sources of information include documentation, system stakeholders and the specifications of similar systems.

- They interact with stakeholders through interview and observation and may use scenarios and prototypes to help with the requirements discovery.

Stakeholders range from system end-users through managers and external stakeholders such as regulators who certify the acceptability of the system.

For example, system stakeholder for a bank ATM include

- Bank customers
- Representatives of other banks
- Bank managers
- Counter staff
- Database administrators
- Security managers
- Marketing department
- Hardware and software maintenance engineers

Requirements sources(stakeholders, domain, systems) can all be represented as system viewpoints, where each viewpoint presents a sub-set of the requirements for the system.

Viewpoints:

- Viewpoints are a way of structuring the requirements to represent the perspectives of different stakeholders. Stakeholders may be classified under different viewpoints.

Types of viewpoint:

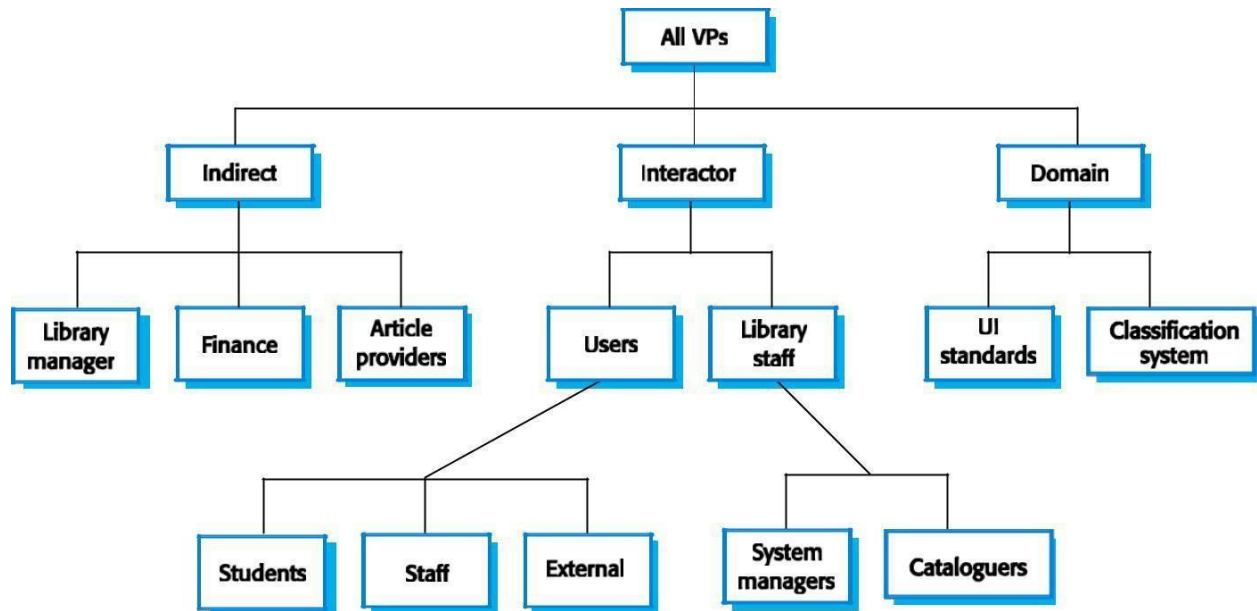
- **Interactor viewpoints**
 - People or other systems that interact directly with the system. These viewpoints provide detailed system requirements covering the system features and interfaces. In an ATM, the customer's and the account database are interactor VPs.

Indirect viewpoints

- – Stakeholders who do not use the system themselves but who influence the requirements. These viewpoints are more likely to provide higher-level organisation requirements and constraints. In an ATM, management and security staff are indirect viewpoints.

Domain viewpoints

- – Domain characteristics and constraints that influence the requirements. These viewpoints normally provide domain constraints that apply to the system. In an ATM, an example would be standards for inter-bank communications.

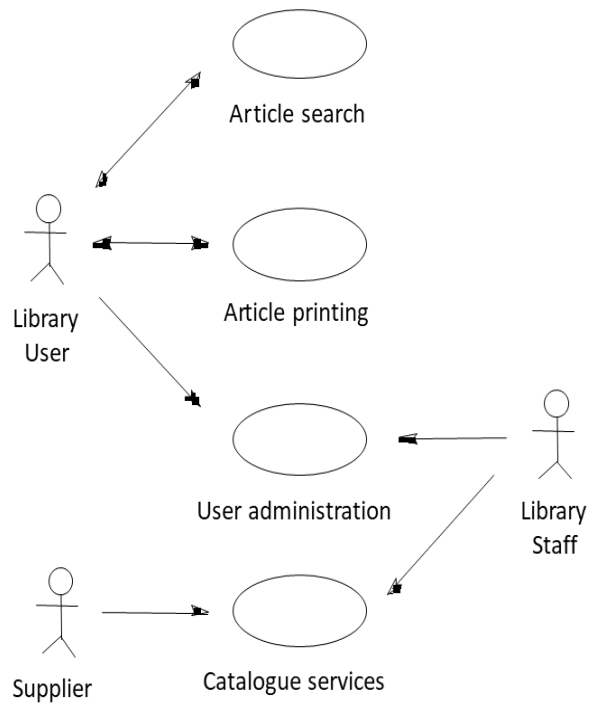


Interviewing

- In formal or informal interviewing, the RE team puts questions to stakeholders about the system that they use and the system to be developed.
- There are two types of interview
- **Closed** interviews where a pre-defined set of questions are answered.
- **Open** interviews where there is no pre-defined agenda and a range of issues are explored with stakeholders.

Scenarios:

- Scenarios are real-life examples of how a system can be used.
- They should include
 - A description of the starting situation;
 - A description of the normal flow of events;
 - A description of what can go wrong;
 - Information about other concurrent activities;
 - A description of the state when the scenario finishes.



10. Discuss about interface specification.

INTERFACE SPECIFICATION

- Most systems must operate with other systems and the operating interfaces must be specified as part of the requirements.
- Three types of interface may have to be defined
- *Procedural interfaces* where existing programs or sub-systems offer a range of services that are accessed by calling interface procedures. These interfaces are sometimes called Application Programming Interfaces (APIs)

--Used for calling the existing programs by the new programs

- *Data structures that are exchanged* that are passed from one sub-system to another. Graphical data models are the best notations for this type of description .

--Provide data passing from one sub-system to another

- *Data representations* that have been established for an existing sub-system.

-- Ordering of bits to match with the existing system

Formal notations are an effective technique for interface specification.

SE UNIT-3 SOLUTIONS

SHORT ANSWER SOLUTIONS

1. Define software design.

Software design is an iterative process through which requirements are translated into a blueprint for constructing the software.

2. What are quality attributes?

Functionality
Usability
Reliability
Performance
Supportability

3. List types of abstraction.

Abstraction: Many levels of abstraction are there.

- At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment.
- At lower levels of abstraction, a more detailed description of the solution is provided.

a. procedural abstraction

b. data abstraction

4. What is modularity?

- Software architecture and design patterns embody **modularity**; software is divided into separately named and addressable components, sometimes called **modules** that are integrated to satisfy problem requirements.
- It has been stated that —modularity is the single attribute of software that allows a program to be intellectually manageable||.

5. Define information hiding.

- The principle of *information hiding* suggests that modules be —characterized by design decision that hides from all others
- Modules should be specified and designed so that information contained within a module is inaccessible to other modules that have no need for such information.

6. Define Coupling and Cohesion.

Coupling is an indication of interconnection among modules in a software structure. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.

A cohesion module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should do just one thing.

7. Define architecture.

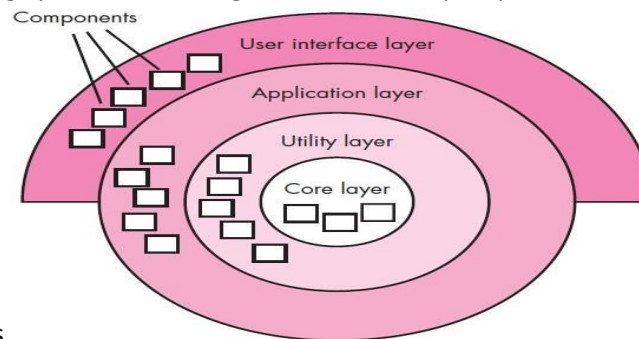
Architecture serves as a **blueprint for a system**. It provides an abstraction to manage the system complexity and establish a communication and coordination mechanism among components.

Or

The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them

8. List types of Layered Architecture.

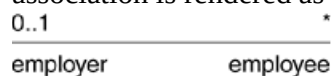
The basic structure of a layered architecture is illustrated in following figure. A number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set. At the outer layer, components service user interface operations. At the inner layer, components perform operating system interfacing. Intermediate layers provide utility services and



application software functions.

9. Define Association and Dependency relationships.

an *association* is a structural relationship among classes that describes a set of links, a link being a connection among objects that are instances of the classes. Aggregation is a special kind of association, representing a structural relationship between a whole and its parts. Graphically, an association is rendered as a solid line, possibly directed, occasionally including a label



a *dependency* is a semantic relationship between two model elements in which a change to one element (the independent one) may affect the semantics of the other element (the dependent one).

Graphically, a dependency is rendered as a dashed line, possibly directed, and occasionally including a label,



10. Define Class and give its graphical representation.

A *class* is a description of a set of objects that share the same attributes, operations, relationships, and semantics. A class implements one or more interfaces. Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations,

Window
origin size
open() close() move() display()

SE UNIT-3 LONG ANSWER SOLUTIONS

1. Explain Quality Attributes.

Quality attributes:

The FURPS quality attributes represent a target for all software design:

- **Functionality** is assessed by evaluating the feature set and capabilities of the program, the generality of the functions that are delivered, and the security of the overall system.
- **Usability** is assessed by considering human factors, overall aesthetics, consistency and documentation.
- **Reliability** is evaluated by measuring the frequency and severity of failure, the accuracy of output results, and the mean – time –to- failure (MTTF), the ability to recover from failure, and the predictability of the program.
- **Performance** is measured by processing speed, response time, resource consumption, throughput, and efficiency
- **Supportability** combines the ability to extend the program (extensibility), adaptability, serviceability- these three attributes represent a more common term maintainability

2. Discuss a) Abstraction b) Modularity in Design Concepts.

Abstraction: Many levels of abstraction are there.

- At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment.
- At lower levels of abstraction, a more detailed description of the solution is provided.
- A **procedural abstraction** refers to a sequence of instructions that have a specific and limited function. The name of procedural abstraction implies these functions, but specific details are suppressed.

ex: open for a door

- A **data abstraction** is a named collection of data that describes a data object.
- In the context of the procedural abstraction *open*, we can define a data abstraction called **door**. Like any data object, the data abstraction for **door** would encompass a set of attributes that describe the door (e.g., door type, swing operation, opening mechanism, weight, dimensions). It follows that the procedural abstraction *open* would make use of information contained in the attributes of the data abstraction **door**.

Modularity:

- Software architecture and design patterns embody **modularity**; software is divided into separately named and addressable components, sometimes called **modules** that are integrated to satisfy problem requirements.
- It has been stated that —modularity is the single attribute of software that allows a program to be intellectually manageable||.
- The —divide and conquer|| strategy- it's easier to solve a complex problem when you break it into manageable pieces. This has important implications with regard to modularity and software. If we subdivide software indefinitely, the effort required to develop it will become negligibly small. The effort to develop an individual software module does decrease as the total number of modules increases. Given the same set of requirements, more modules means smaller individual size. However, as the number of modules grows, the effort associated with integrating the modules also grows
- Under modularity or over modularity should be avoided. We modularize a design so that development can be more easily planned; software increment can be defined and delivered; changes can be more easily accommodated; testing and debugging can be conducted more efficiently, and long-term maintenance can be conducted without serious side effects.

3.Explain a) Refinement b) Refactoring in Design Concepts.

Refinement:

- Stepwise refinement is a top- down design strategy originally proposed by Niklaus wirth. A program is development by successively refining levels of procedural detail. A hierarchy is development by decomposing a macroscopic statement of function in a step wise fashion until programming language statements are reached.
- Refinement is actually a process of elaboration. We begin with a statement of function that is defined at a high level of abstraction. That is, the statement describes function or information conceptually but provides no information about the internal workings of the function or the internal structure of the data. Refinement causes the designer to elaborate on the original statement, providing more and more detail as each successive refinement occurs.
- Abstraction and refinement are complementary concepts. Abstraction enables a designer to specify procedure and data and yet suppress low-level details. Refinement helps the designer to reveal low-level details as design progresses. Both concepts aid the designer in creating a complete design model as the design evolves.

Refactoring :

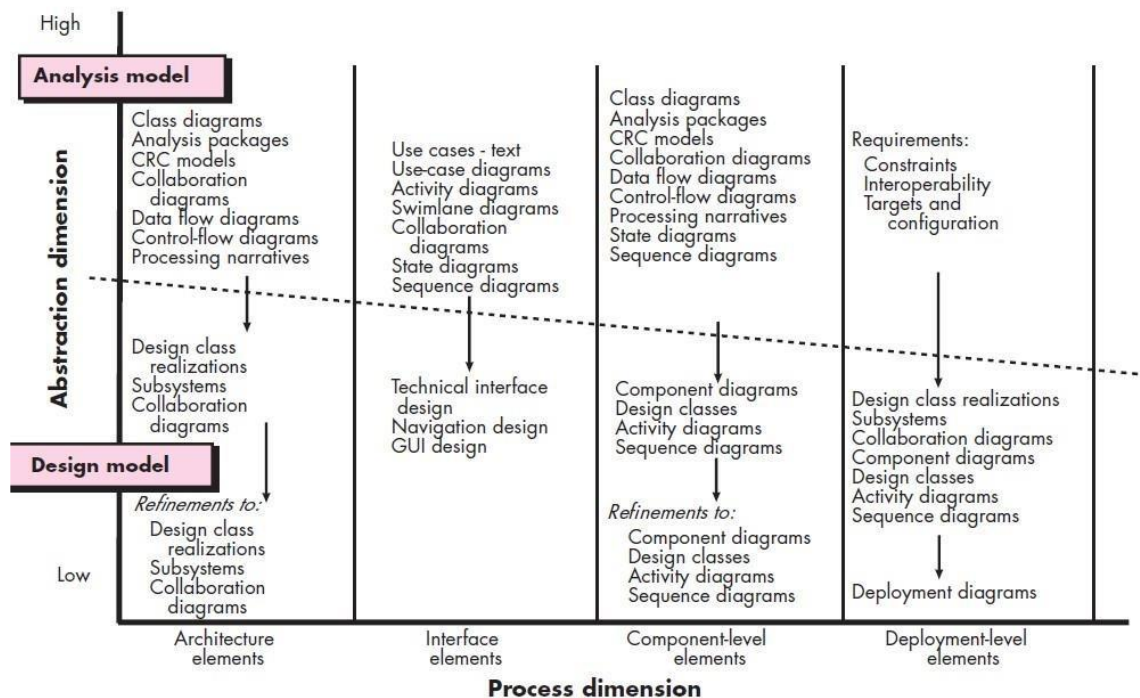
- Refactoring is a reorganization technique that simplifies the design of a component without changing its function or behavior. Fowler defines refactoring in the following manner: —refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure.||
- When software is refactored, the existing design is examined for redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failure that can be corrected to yield a better design. The designer may decide that the component should be refactored into 3 separate components, each exhibiting high cohesion. The result will be software that is easier to integrate, easier to test, and easier to maintain

4. Discuss about the design model.

THE DESIGN MODEL:

- The design model can be viewed into 2 different dimensions.
- The process dimension indicates the evolution of the design model as design tasks are executed as a part of the software process.
- The abstraction dimension represents the level of detail as each element of the analysis model is transformed into a design equivalent and then refined iteratively.

- The elements of the design model use many of the same UML diagrams that were used in the analysis model. The difference is that these diagrams are refined and elaborated as a path of design; more implementation- specific detail is provided, and architectural structure and style, components that reside within the architecture, and the interface between the components and with the outside world are all emphasized.
- It is important to mention however, that model elements noted along the horizontal axis are not always developed in a sequential fashion..



• Data design elements:

Data design sometimes referred to as data architecting creates a model of data and/or information that is represented at a high level of abstraction. This data model is then refined into progressively more implementation-specific representations that can be processed by the computer-based system.

- The structure of data has always been an important part of software design.

Architectural Design Elements

- The **architectural design** for software is the equivalent to the floor plan of a house. The floor plan depicts the overall layout of the rooms; their size, shape, and relationship to one another; and the doors and windows that allow movement into and out of the rooms. Architectural design elements give us an overall view of the software.

Interface design elements:

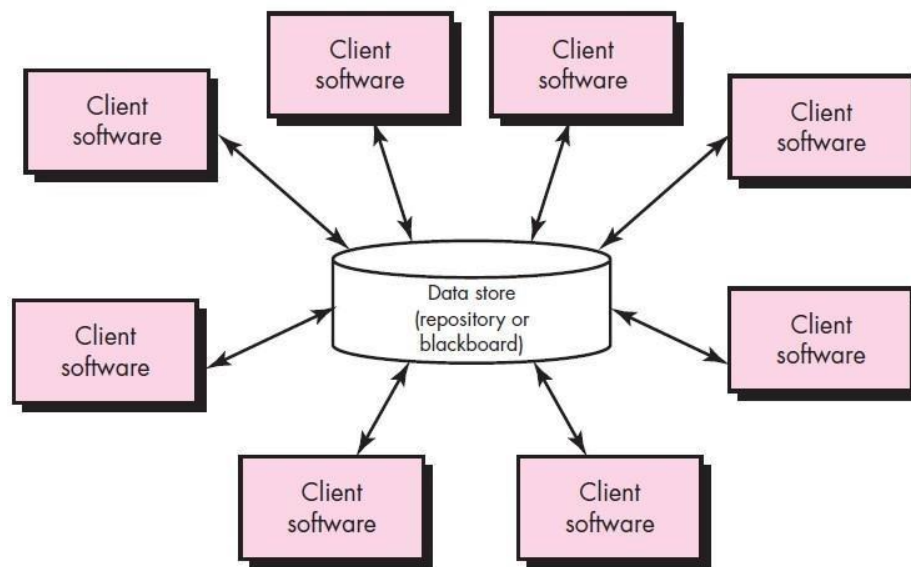
- *The interface design for software is the equivalent to a set of detailed drawings for the doors, windows, and external utilities of a house*

There are 3 important elements of interface design:

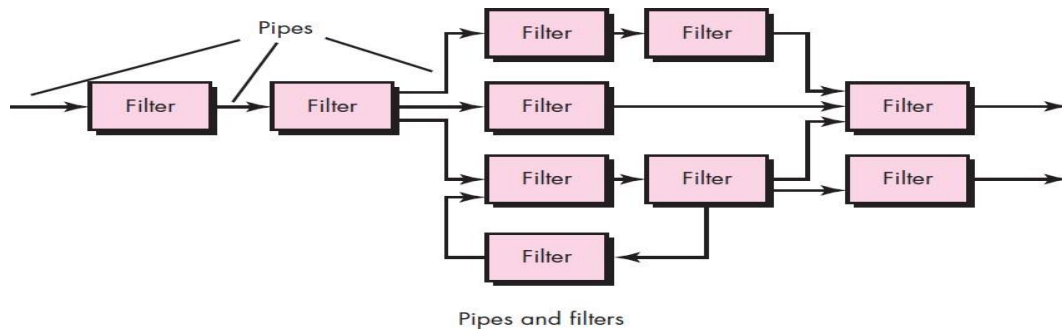
- The user interface(UI);
- External interfaces to other systems, devices, networks, or other produces or consumers of information; and
- Internal interfaces between various design components.
- These interface design elements allow the software to communicated externally and enable internal communication and collaboration among the components that populate the software architecture.
- **Component-Level Design Elements**
 - The component-level design for software is the equivalent to a set of detailed drawings for each room in a house. These drawings depict wiring and plumbing within each room, the location of electrical receptacles and wall switches, sinks, showers, tubs, drains, cabinets, and closets.
 - The component-level design for software fully describes the internal detail of each software component. To accomplish this, the component-level design defines data structures for all local data objects and algorithmic detail for all processing that occurs within a component and an interface that allows access to all component operations.
- **Deployment-Level Design Elements**
 - Deployment-level design elements indicate how software functionality and subsystems will be allocated within the physical computing environment that will support the software.
 - Deployment diagrams begin in descriptor form, where the deployment environment is described in general terms. Later, instance form is used and elements of the configuration are explicitly described.

5.Explain a) Data-Centered Architecture b) Data-flow Architecture.

- **Data-centered architectures.** A data store (e.g., a file or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store. The following figure illustrates a typical data-centered style. Client software accesses a central repository. In some cases the data repository is passive. Data-centered architectures promote *integrability*.

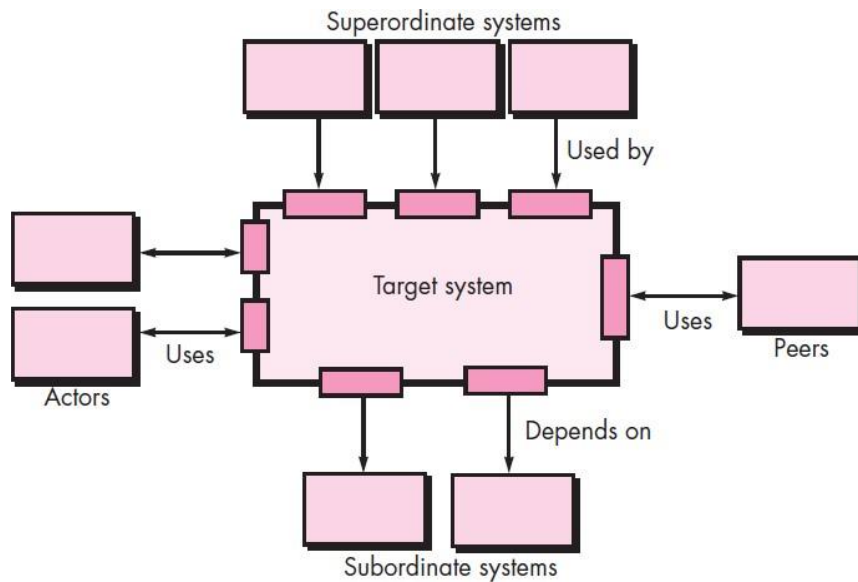


Data-flow architectures. This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data. A pipe-and-filter pattern shown in following figure. It has a set of components, called *filters*, connected by *pipes* that transmit data from one component to the next. Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output of a specified form. However, the filter does not require knowledge of the Workings of its neighboring filters.



6.Explain Architectural Context Diagram.

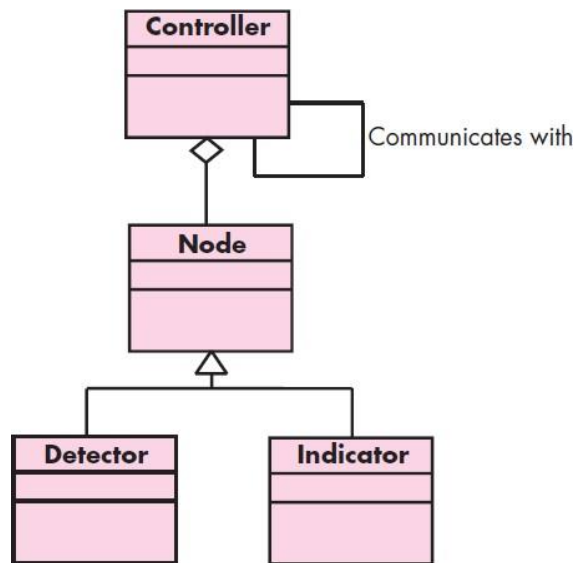
- At the architectural design level, a software architect uses an **architectural context diagram(ACD)** to model the manner in which software interacts with entities external to its boundaries. The generic structure of the architectural context diagram is illustrated in following figure. Referring to the figure, systems that interoperate with the *target system* (the system for which an architectural design is to be developed) are represented as
 - Superordinate systems**—those systems that use the target system as part of some higher-level processing scheme.
 - Subordinate systems**—those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.
 - Peer-level systems**—those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system).
 - Actors**—entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.



7.Explain Archetype with diagram.

- **Defining Archetypes**
- An **archetype** is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, a relatively small set of archetypes is required to design even relatively complex systems. The target system architecture is composed of these archetypes, which represent stable elements of the architecture but may be instantiated many different ways based on the behavior of the system.
- The following archetypes can be used :
- **Node.** Represents a cohesive collection of input and output elements of the home security function. For example a node might be comprised of (1) various sensors and (2) a variety of alarm (output) indicators.
- **Detector.** An abstraction that encompasses all sensing equipment that feeds information into the target system

- **Indicator.** An abstraction that represents all mechanisms (e.g., alarm siren, flashing lights, bell) for indicating that an alarm condition is occurring.
- **Controller.** An abstraction that depicts the mechanism that allows the arming or disarming of a node. If controllers reside on a network, they have the ability to communicate with one another.



8.Explain a) Things b) Common Mechanisms in Conceptual Model in UML.

Things in the UML

There are four kinds of things in the UML:

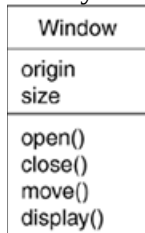
1. Structural things
2. Behavioral things
3. Grouping things
4. Annotational things

These things are the basic object-oriented building blocks of the UML. You use them to write well-formed models.

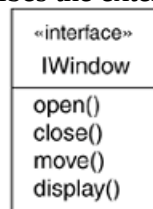
Structural Things

Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. Collectively, the structural things are called *classifiers*.

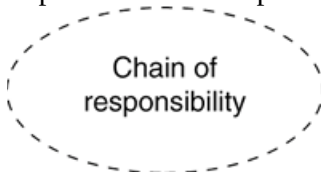
A *class* is a description of a set of objects that share the same attributes, operations, relationships, and semantics. A class implements one or more interfaces. Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations,



An *interface* is a collection of operations that specify a service of a class or component. An interface therefore describes the externally visible behavior of that element.



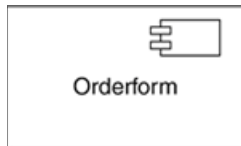
A *collaboration* defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements.



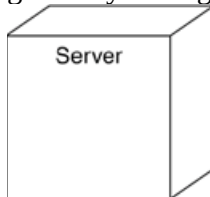
A *use case* is a description of sequences of actions that a system performs that yield observable results of value to a particular actor.



A *component* is a modular part of the system design that hides its implementation behind a set of external interfaces



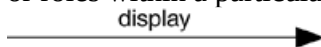
A *node* is a physical element that exists at run time and represents a computational resource, generally having at least some memory and, often, processing capability.



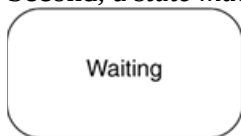
Behavioral Things

Behavioral things are the dynamic parts of UML models. These are the verbs of a model, representing behavior over time and space. In all, there are three primary kinds of behavioral things.

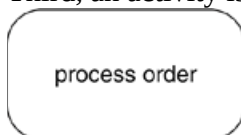
First, an *interaction* is a behavior that comprises a set of messages exchanged among a set of objects or roles within a particular context to accomplish a specific purpose



Second, a *state machine* is a behavior that specifies the sequences of states an object or an

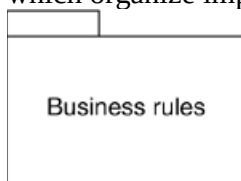


Third, an *activity* is a behavior that specifies the sequence of steps a computational process performs.



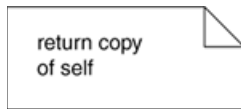
Grouping Things Grouping things are the organizational parts of UML models. These are the boxes into which a model can be decomposed. There is one primary kind of grouping thing, namely, packages.

A *package* is a general-purpose mechanism for organizing the design itself, as opposed to classes, which organize implementation constructs



Annotational Things

Annotational things are the explanatory parts of UML models. These are the comments you may apply to describe, illuminate, and remark about any element in a model. There is one primary kind of annotational thing, called a note.



Common Mechanisms in the UML

A building is made simpler and more harmonious by the conformance to a pattern of common features.

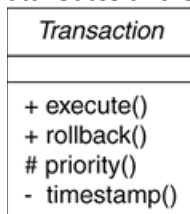
1. Specifications
2. Adornments
3. Common divisions
4. Extensibility mech

Specifications

The UML is more than just a graphical language. Rather, behind every part of its graphical notation there is a specification that provides a textual statement of the syntax and semantics of that building block.

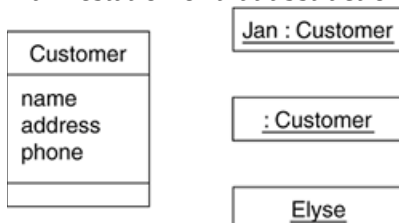
Adornments Most elements in the UML have a unique and direct graphical notation that provides a visual representation of the most important aspects of the element.

A class's specification may include other details, such as whether it is abstract or the visibility of its attributes and operations.



Common Divisions In modeling object-oriented systems, the world often gets divided in several ways.

First, there is the division of class and object. A class is an abstraction; an object is one concrete manifestation of that abstraction. In the UML, you can model classes as well as objects, as shown in



Interfaces and Implementations



Extensibility Mechanisms

UML's extensibility mechanisms include

Stereotypes

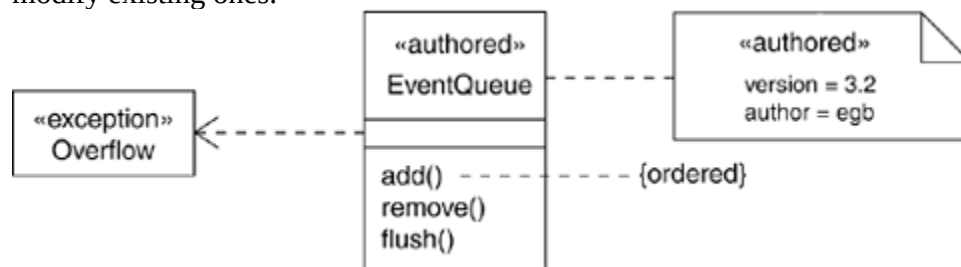
Tagged values

Constraints

A **stereotype** extends the vocabulary of the UML, allowing you to create new kinds of building blocks that are derived from existing ones but that are specific to your problem

A **tagged value** extends the properties of a UML stereotype, allowing you to create new information in the stereotype's specification.

A **constraint** extends the semantics of a UML building block, allowing you to add new rules or modify existing ones.



9.Explain a) Class Diagram b) Use Case Diagram.

Class Diagram

The class diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes

It shows the attributes, classes, functions, and relationships to give an overview of the software system. It constitutes class names, attributes, and functions in a separate compartment that helps in software development.

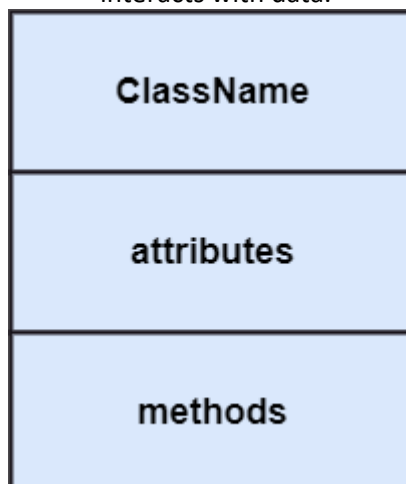
Purpose of Class Diagrams

The main purpose of class diagrams is to build a static view of an application. It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages. It is one of the most popular UML diagrams. Following are the purpose of class diagrams given below:

1. It analyses and designs a static view of an application.
2. It describes the major responsibilities of a system.
3. It is a base for component and deployment diagrams.
4. It incorporates forward and reverse engineering.

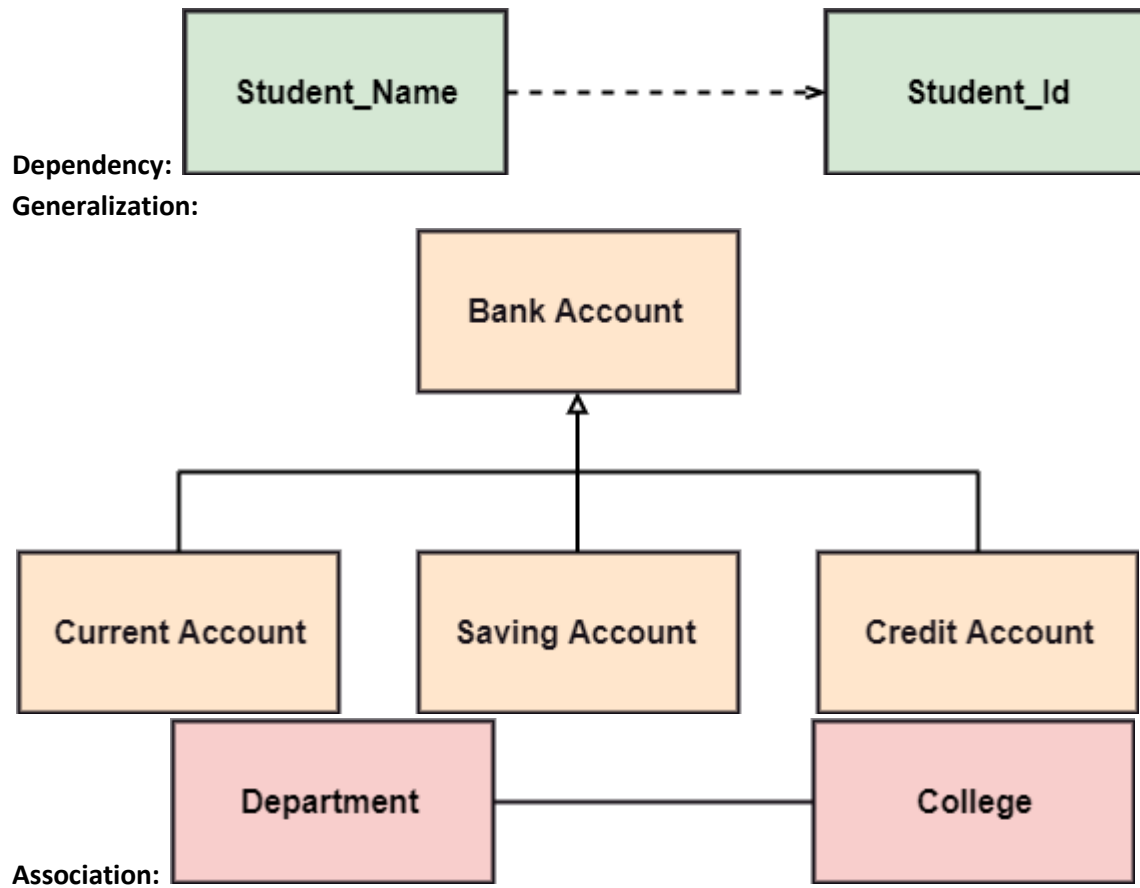
The class diagram is made up of three sections:

- **Upper Section:** The upper section encompasses the name of the class. Some of the following rules that should be taken into account while representing a class are given below:
 1. Capitalize the initial letter of the class name.
 2. Place the class name in the center of the upper section.
 3. A class name must be written in bold format.
 4. The name of the abstract class should be written in italics format.
- **Middle Section:** The middle section constitutes the attributes, which describe the quality of the class. The attributes are written along with its visibility factors, which are public (+), private (-), protected (#), and package (~).
- **Lower Section:** The lower section contains methods or operations. The methods are represented in the form of a list, where each method is written in a single line. It demonstrates how a class interacts with data.



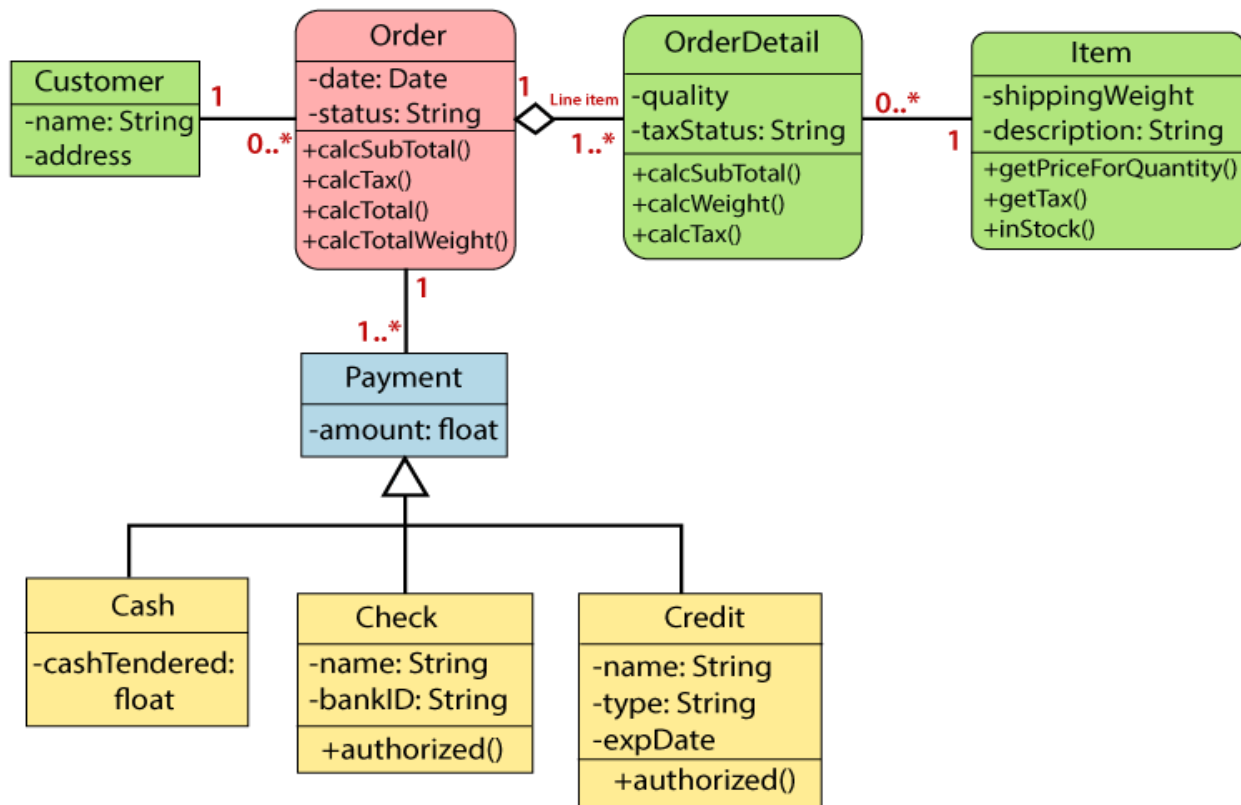
Relationships

In UML, relationships are of three types:



Class Diagram Example

A class diagram describing the sales order system is given below.



Use Case Diagram

A use case diagram is used to represent the dynamic behavior of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles a system.

Purpose of Use Case Diagrams

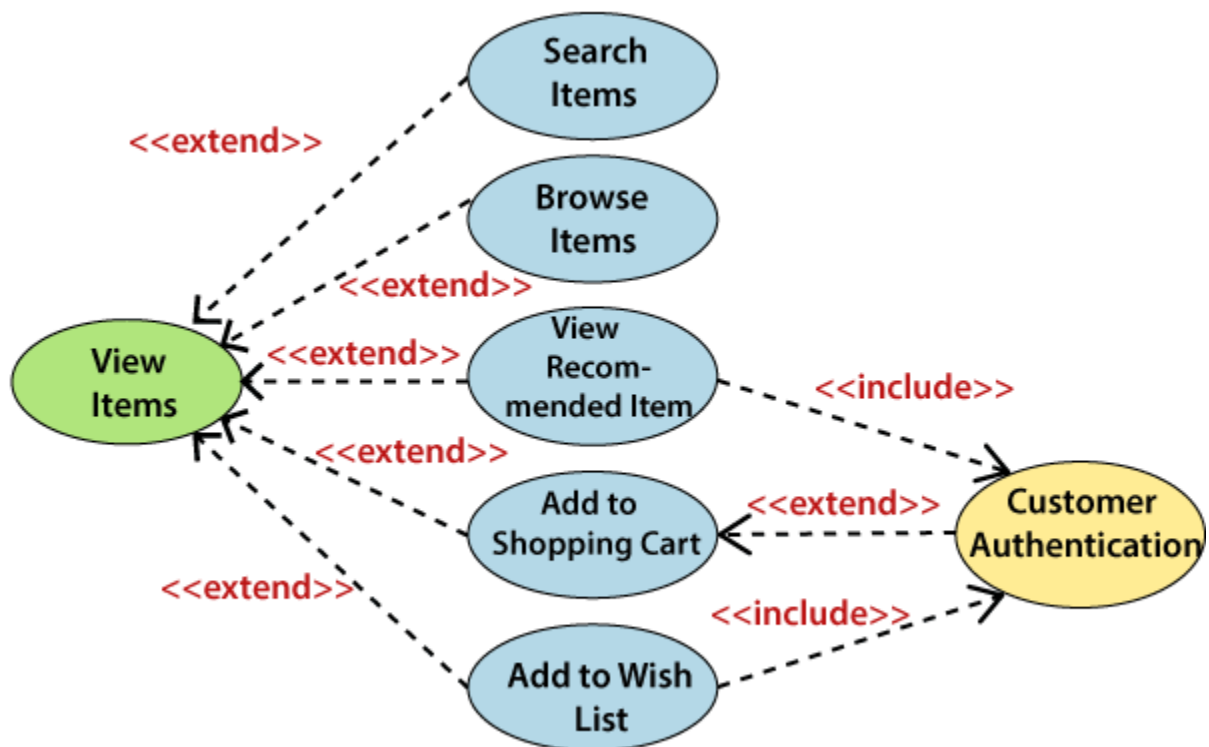
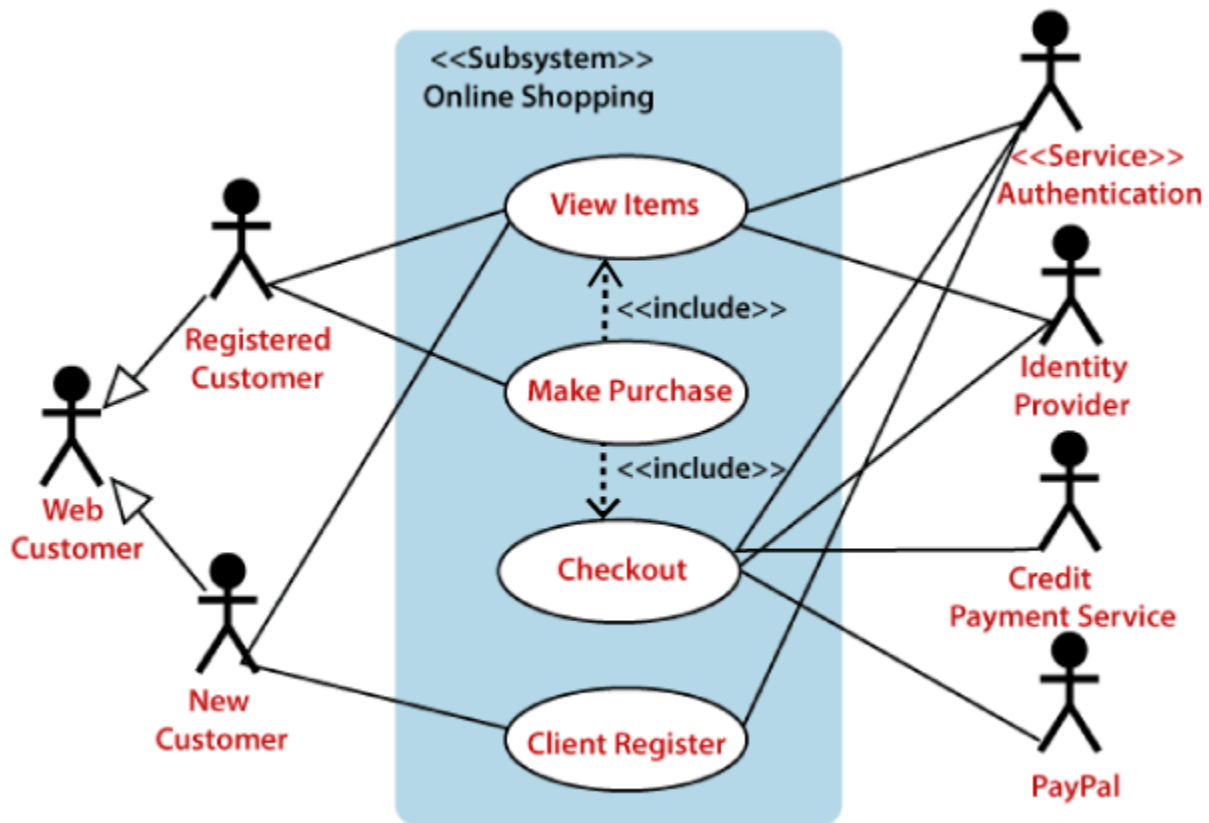
Following are the purposes of a use case diagram given below:

1. It gathers the system's needs.
2. It depicts the external view of the system.
3. It recognizes the internal as well as external factors that influence the system.
4. It represents the interaction between the actors.

A use case diagram depicting the Online Shopping website is given below.

Here the Web Customer actor makes use of any online shopping website to purchase online. The top-level uses are as follows; View Items, Make Purchase, Checkout, Client Register. The **View Items** use case is utilized by the customer who searches and view products. The **Client Register** use case allows the customer to register itself with the website for availing gift vouchers, coupons, or getting a private sale

invitation. It is to be noted that the **Checkout** is an included use case, which is part of **Making Purchase**, and it is not available by itself.



10.Explain a) Sequence Diagram b) Component Diagram.

Sequence Diagram

The sequence diagram represents the flow of messages in the system and is also termed as an event diagram. It helps in envisioning several dynamic scenarios. It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelines took part at the run time. In UML, the lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page. It incorporates the iterations as well as branching.

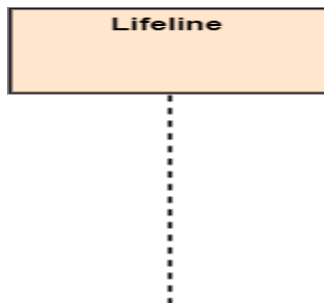
Purpose of a Sequence Diagram

1. To model high-level interaction among active objects within a system.
2. To model interaction among objects inside a collaboration realizing a use case.
3. It either models generic interactions or some certain instances of interaction.

Notations of a Sequence Diagram

Lifeline

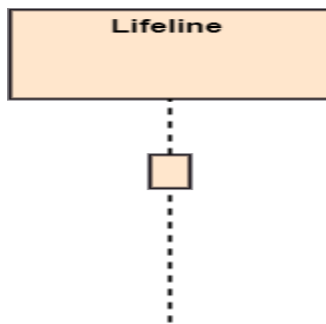
An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.



Actor



Activation

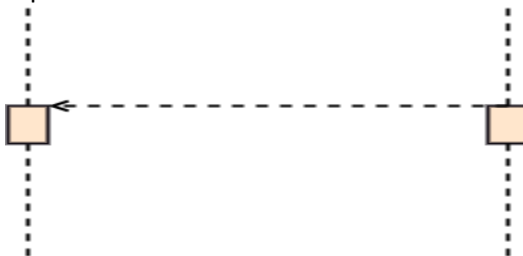


Messages

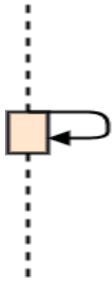
The messages depict the interaction between the objects and are represented by arrows. They are in the sequential order on the lifeline. The core of the sequence diagram is formed by messages and lifelines.

Following are types of messages enlisted below:

- **Call Message:** It defines a particular communication between the lifelines of an interaction, which represents that the target lifeline has invoked an operation.
- **Return Message:** It defines a particular communication between the lifelines of interaction that represent the flow of information from the receiver of the corresponding caller message.



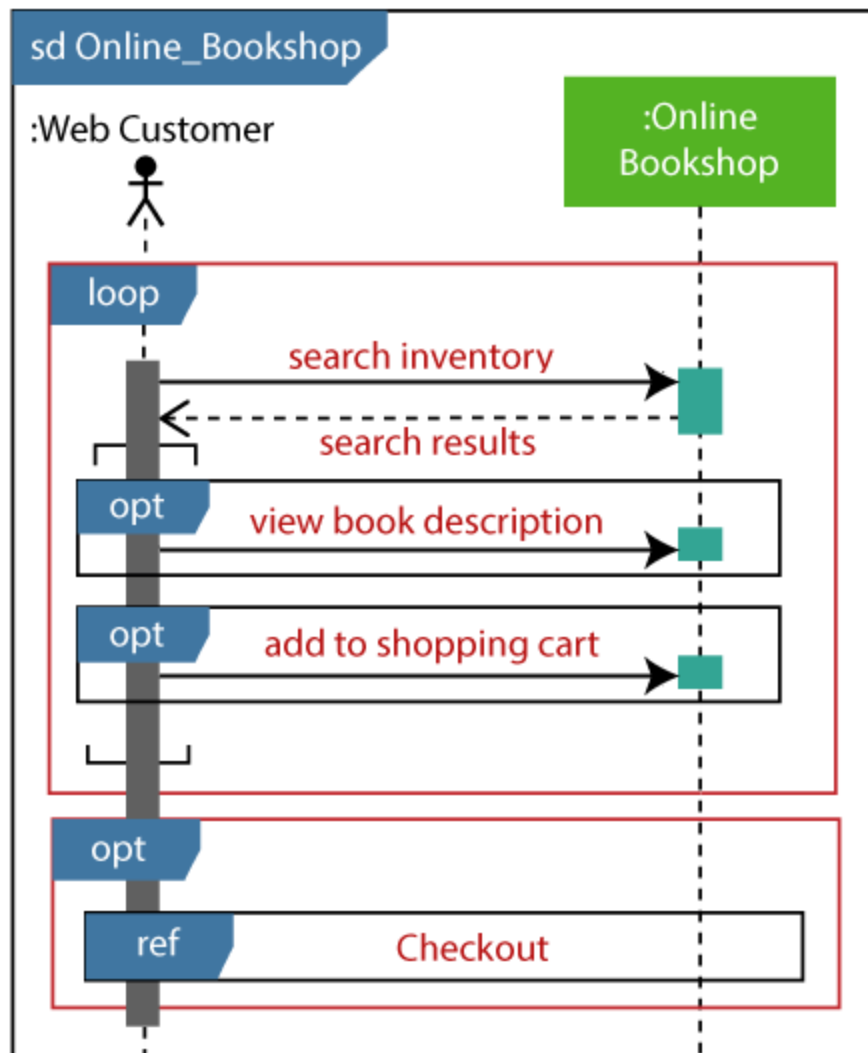
- **Self Message:** It describes a communication, particularly between the lifelines of an interaction that represents a message of the same lifeline, has been invoked.



Example of a Sequence Diagram

An example of a high-level sequence diagram for online bookshop is given below.

Any online customer can search for a book catalog, view a description of a particular book, add a book to its shopping cart, and do checkout.



Benefits of a Sequence Diagram

1. It explores the real-time application.
2. It depicts the message flow between the different objects.
3. It has easy maintenance.
4. It is easy to generate.
5. Implement both forward and reverse engineering.
6. It can easily update as per the new change in the system.

The drawback of a Sequence Diagram

1. In the case of too many lifelines, the sequence diagram can get more complex.

2. The incorrect result may be produced, if the order of the flow of messages changes.
3. Since each sequence needs distinct notations for its representation, it may make the diagram more complex.
4. The type of sequence is decided by the type of message.

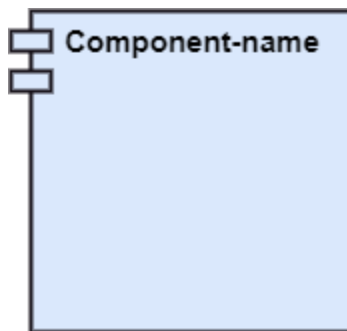
Component Diagram

A component diagram is used to break down a large object-oriented system into the smaller components, so as to make them more manageable. It models the physical view of a system such as executables, files, libraries, etc. that resides within the node.

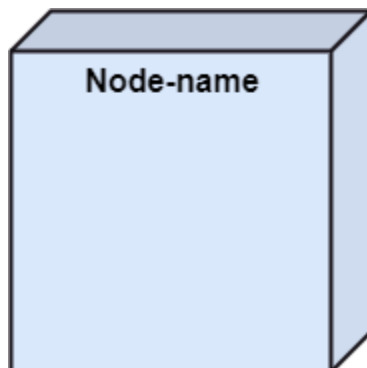
It visualizes the relationships as well as the organization between the components present in the system. It helps in forming an executable system. A component is a single unit of the system, which is replaceable and executable.

Notation of a Component Diagram

a) A component



b) A node



Purpose of a Component Diagram

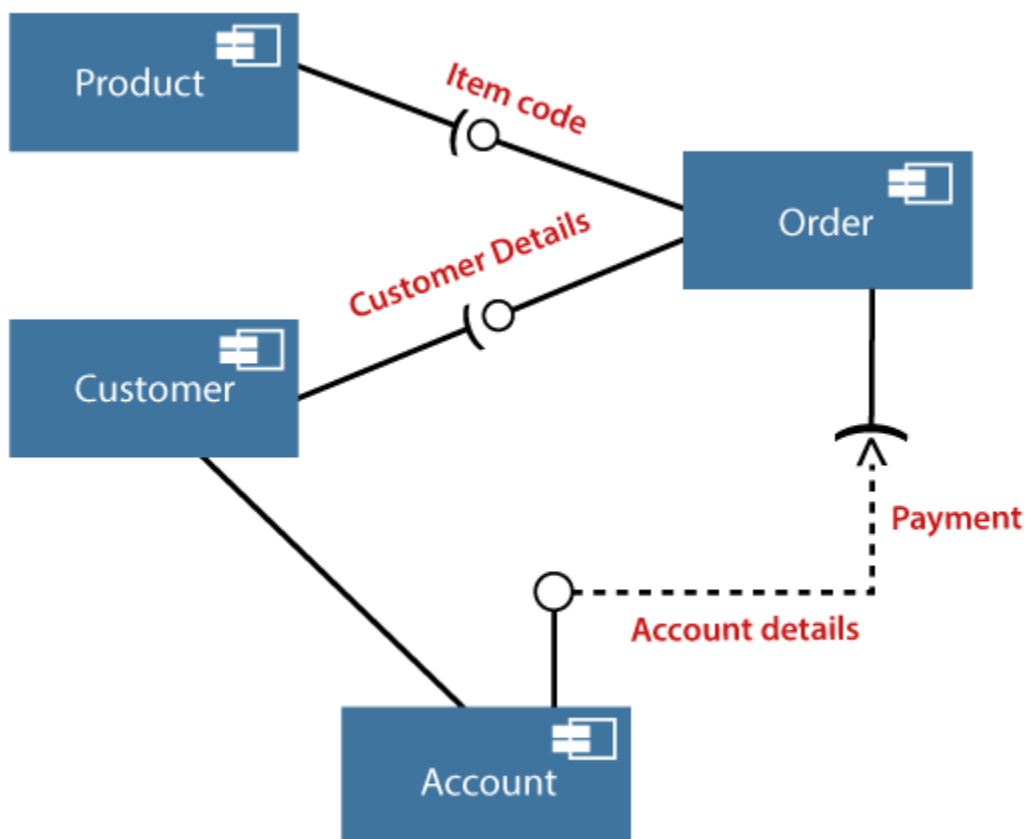
Since it is a special kind of a UML diagram, it holds distinct purposes. It describes all the individual components that are used to make the functionalities, but not the functionalities of the system. It visualizes the physical components inside the system. The components can be a library, packages, files, etc.

When to use a Component Diagram?

It represents various physical components of a system at runtime. It is helpful in visualizing the structure and the organization of a system. It describes how individual components can together form a single system. Following are some reasons, which tells when to use component diagram:

1. To divide a single system into multiple components according to the functionality.
2. To represent the component organization of the system.

Example of a Component Diagram



Where to use Component Diagrams?

The component diagram is a special purpose diagram, which is used to visualize the static implementation view of a system. It represents the physical components of a system, or we can say it portrays the organization of the components inside a system. The components, such as libraries, files, executables, etc. are first needed to be organized before the implementation

SE UNIT-4 SOLUTIONS

SHORT ANSWER SOLUTIONS

1. Define the terms a) Testing b) Debugging.

Testing: Testing is the process of verifying and validating that a software or application is bug-free, meets the technical requirements as guided by its design and development, and meets the user requirements effectively and efficiently by handling all the exceptional and boundary cases.

Debugging: Debugging is the process of fixing a bug in the software. It can be defined as identifying, analyzing, and removing errors.

2. Define Unit Testing.

Unit testing focuses verification effort on the smallest unit of software design. The unit test focuses on the internal processing logic and data structures within the boundaries of a component. This type of testing can be conducted in parallel for multiple components

3. What is Integration Testing?

- Integration testing is a systematic technique for constructing the software architecture while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit-tested components and build a program structure that has been dictated by design.

4.What is Alpha testing?

- The **alpha test** is conducted at the developer's site by a representative group of end users. The software is used in a natural setting with the developer "looking over the shoulder" of the users and recording errors and usage problems. Alpha tests are conducted in a controlled environment.

5.What is Beta Testing?

- The **beta test** is conducted at one or more end-user sites. Unlike alpha testing, the developer generally is not present. Therefore, the beta test is a "live" application of the software in an environment that cannot be controlled by the developer. The customer records all problems that are encountered during beta testing and reports these to the developer at regular intervals.

6.Define White Box Testing Strategy.

- [White-box testing](#) is a testing technique which checks the internal functioning of the system. In this method, testing is based on coverage of code statements, branches, paths or conditions. White-Box testing is considered as low-level testing. It is also called glass box, transparent box, clear box or code base testing. The white-box Testing method assumes that the path of the logic in a unit or program is known.

7.Define Black Box Testing Strategy.

- In [Black-box testing](#), a tester doesn't have any information about the internal working of the software system. Black box testing is a high level of testing that focuses on the behavior of the software. It involves testing from an external or end-user perspective. Black box testing can be applied to virtually every level of software testing: unit, integration, system, and acceptance.

8.Define Software Reliability.

Reliability: The extent to which program can be expected to perform its intended function with required precision.

9.What is Smoke Testing?

Smoke testing is an integration testing approach that is commonly used when product software is developed. It is designed as a pacing mechanism for time-critical projects, allowing the software team to assess the project on a frequent basis.

10.List the types of Testing.

Unit testing

Integration testing

System testing

Validation testing

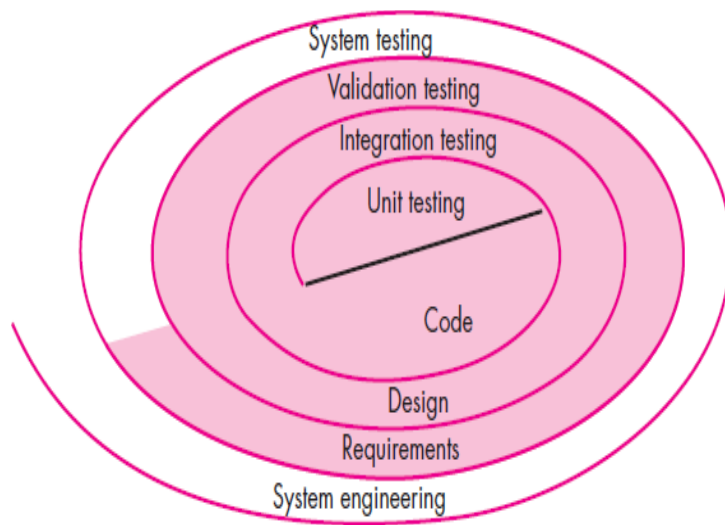
Acceptance testing

SE UNIT-4

LONG ANSWER SOLUTIONS

1. Elaborate the Test Strategies for Conventional Software.

- The software process may be viewed as the spiral illustrated in following figure. Initially, system engineering defines the role of software and leads to software requirements analysis, where the information domain, function, behavior, performance, constraints, and validation criteria for software are established. Moving inward along the spiral, you come to design and finally to coding. To develop computer software, you spiral inward (counter clockwise) along streamlines that decrease the level of abstraction on each turn.



A strategy for software testing may also be viewed in the context of the spiral.

Unit testing begins at the vortex of the spiral and concentrates on each unit of the software as implemented in source code.

Testing progresses by moving outward along the spiral to **integration testing**, where the focus is on design and the construction of the software architecture.

Taking another turn outward on the spiral, you encounter **validation testing**, where requirements established as part of requirements modeling are validated against the software that has been constructed.

Finally, you arrive at **system testing**, where the software and other system elements are tested as a whole. Initially, tests focus on each component individually, ensuring that it functions properly as a unit.

Hence, the name **unit testing**. Unit testing makes heavy use of testing techniques that exercise specific paths in a component's control structure to ensure complete coverage and maximum error detection.

Next, components must be assembled or integrated to form the complete software package.

Integration testing addresses the issues associated with the dual problems of verification and program construction. Test case design techniques that focus on inputs and outputs are more prevalent during integration, although techniques that exercise specific program paths may be used to ensure coverage of major control paths. After the software has been integrated (constructed), a set of *high-order tests* is conducted. Validation criteria must be evaluated. **Validation testing** provides final assurance that software meets all informational, functional, behavioral, and performance requirements.

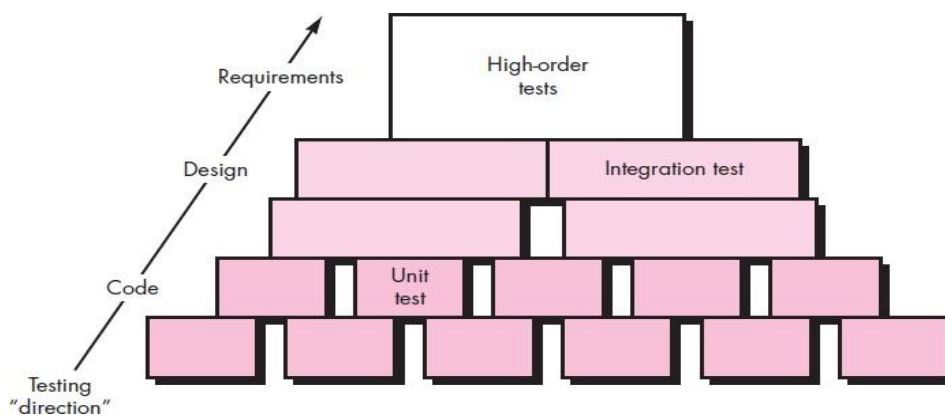


Fig : Software testing steps

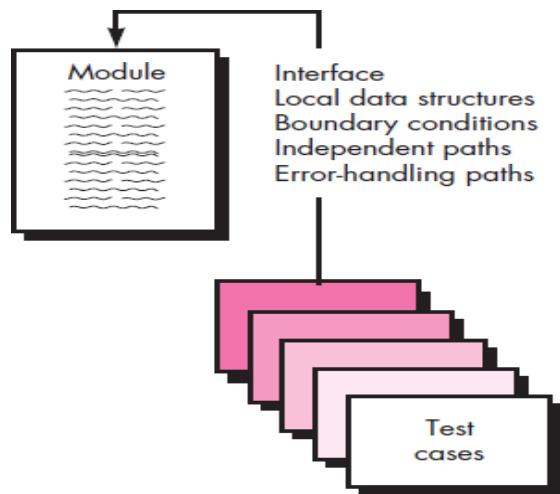
The last high-order testing step falls outside the boundary of software engineering and into the broader context of computer system engineering. Software, once validated, must be combined with other system elements (e.g., hardware, people, databases). **System testing** verifies that all elements mesh properly and that overall system function/performance is achieved.

2. Explain about Unit Testing.

Unit testing focuses verification effort on the smallest unit of software design. The unit test focuses on the internal processing logic and data structures within the boundaries of a component. This type of testing can be conducted in parallel for multiple components.

- **Unit-test considerations.** Unit tests are illustrated schematically in following figure. The module **interface** is tested to ensure that information properly flows into and out of the program unit under test. **Local data structures** are examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution. All **independent paths** through the control structure are exercised to ensure that all statements in a module have been executed at

least once. **Boundary conditions** are tested to ensure that the module operates properly at boundaries established to limit or restrict processing. And finally, all **error-handling paths** are tested.



- **Unit-test procedures.** Unit testing is normally considered as an adjunct to the coding step. The design of unit tests can occur before coding begins or after source code has been generated.
- The unit test environment is illustrated in following figure. In most applications a **driver** is nothing more than a “main program” that accepts test case data, passes such data to the component (to be tested), and prints relevant results. **Stubs** serve to replace modules that are subordinate (invoked by) the component to be tested.

3. Discuss about regression testing and smoke testing.

Regression testing.

Regression testing is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects. Regression testing helps to ensure that changes do not introduce unintended behavior or additional errors.

Regression testing may be conducted manually, by re-executing a subset of all test cases or using automated **capture/playback** tools. **Capture/playback tools** enable the software engineer to capture test cases and results for subsequent playback and comparison. The **regression test suite** (the subset of tests to be executed) contains **three** different classes of test cases:

- A representative sample of tests that will exercise all software functions.
- Additional tests that focus on software functions that are likely to be affected by the change.
- Tests that focus on the software components that have been changed.

Smoke testing:

- **Smoke testing** is an integration testing approach that is commonly used when product software is developed. It is designed as a pacing mechanism for time-critical projects, allowing the software team to assess the project on a frequent basis. In essence, the smoke- testing approach encompasses the following activities:
- Software components that have been translated into code are integrated into a *build*. A build includes all data files, libraries, reusable modules, and engineered components that are required to implement one or more product functions.
- A series of tests is designed to expose errors that will keep the build from properly performing its function. The intent should be to uncover “showstopper” errors that have the highest likelihood of throwing the software project behind schedule.
- The build is integrated with other builds, and the entire product (in its current form) is smoke tested daily. The integration approach may be top down or bottom up.
- Smoke testing provides a number of **benefits** when it is applied on complex, time critical software projects:
- ***Integration risk is minimized***
- ***The quality of the end product is improved.***
- ***Error diagnosis and correction are simplified***
- ***Progress is easier to assess.***

4. Discuss about Validation Testing.

- Validation testing begins at the culmination of integration testing, when individual components have been exercised, the software is completely assembled as a package, and interfacing errors have been uncovered and corrected.
- Validation can be defined in many ways, but a simple definition is that validation succeeds when software functions in a manner that can be reasonably expected by the customer.

Validation-Test Criteria

Software validation is achieved through a series of tests that demonstrate conformity with requirements. After each validation test case has been conducted, one of two possible conditions exists: (1) The function or performance characteristic conforms to specification and is accepted or (2) A deviation from specification is uncovered and a deficiency list is created.

Configuration Review

- An important element of the validation process is a **configuration review**. The intent of the review is to ensure that all elements of the software configuration have been properly developed, are cataloged, and have the necessary detail to bolster the support activities. The configuration review, sometimes called an **audit**

Alpha and Beta Testing

When custom software is built for one customer, a series of **acceptance tests** are conducted to enable the customer to validate all requirements.

- The **alpha test** is conducted at the developer's site by a representative group of end users. The software is used in a natural setting with the developer "looking over the shoulder" of the users and recording errors and usage problems. Alpha tests are conducted in a controlled environment.
- The **beta test** is conducted at one or more end-user sites. Unlike alpha testing, the developer generally is not present. Therefore, the beta test is a "live" application of the software in an environment that cannot be controlled by the developer. The customer records all problems that are encountered during beta testing and reports these to the developer at regular intervals.
- A variation on beta testing, called **customer acceptance testing**, is sometimes performed when custom software is delivered to a customer under contract. The customer performs a series of specific tests in an attempt to uncover errors before accepting the software from the developer.

5.Explain System Testing.

SYSTEM TESTING

- *System testing* is actually a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that system elements have been properly integrated and perform allocated functions.

Recovery Testing

- *Recovery testing* is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed. If recovery is automatic (performed by the system itself), reinitialization, checkpointing mechanisms, data recovery, and restart are evaluated for correctness. If recovery requires human intervention, the mean-time-to-repair (MTTR) is evaluated to determine whether it is within acceptable limits.

Security Testing

- *Security testing* attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration. During security testing, the tester plays the role(s) of the individual who desires to penetrate the system. Good security testing will ultimately penetrate a system. The role of the system designer is to make penetration cost more than the value of the information that will be obtained.

Stress Testing

Stress tests are designed to confront programs with abnormal situations. *Stress testing* executes a system in a manner that demands resources in abnormal quantity, frequency, or volume. For example, (1) special tests may be designed that generate ten interrupts per second, when one or two is the average rate, (2) input data rates may be increased by an order of magnitude to determine how input functions will respond, (3) test cases that require maximum memory or other resources are executed, (4) test cases that may cause thrashing in a virtual operating system are designed, (5) test cases that may cause excessive hunting for disk-resident data are created.

Performance Testing

- Performance testing is designed to test the run-time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as tests are conducted. Performance tests are often coupled with stress testing and usually require both hardware and software instrumentation.

Deployment Testing

- *Deployment testing*, sometimes called **configuration testing**, exercises the software in each environment in which it is to operate. In addition, deployment testing examines all installation procedures and specialized installation software (e.g., “installers”) that will be used by customers, and all documentation that will be used to introduce the software to end users.

6. Discuss the Art of Debugging with neat Diagram.

THE ART OF DEBUGGING

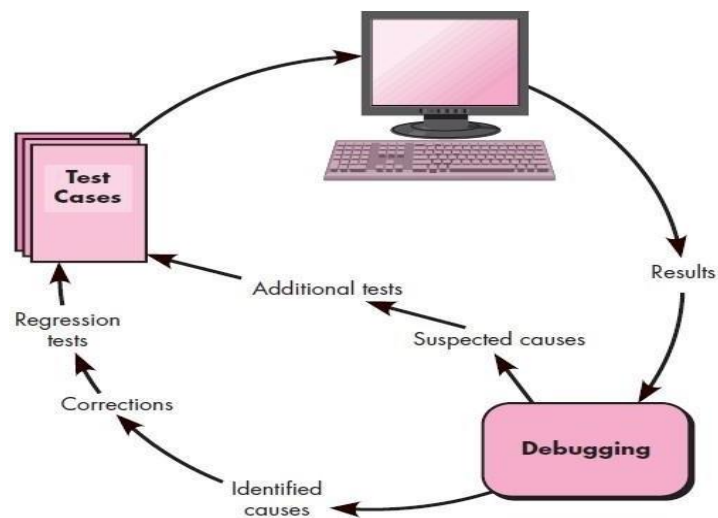
Debugging occurs as a consequence of successful testing. That is, when a test case uncovers an error, debugging is the process that results in the removal of the error. Although debugging can and should be an orderly process, it is still very much an art.

The Debugging Process

- Debugging is not testing but often occurs as a consequence of testing. Referring to the following figure, the debugging process begins with the execution of a test case.. The debugging process attempts to match symptom with cause, thereby leading to error correction.
- The debugging process will usually have one of **two** outcomes:

1.the cause will be found and corrected or

2.the cause will not be found.



A few characteristics of bugs provide some clues:

- The symptom and the cause may be geographically remote. That is, the symptom may appear in one part of a program, while the cause may actually be located at a site that is far removed. Highly coupled components exacerbate this situation.
- The symptom may disappear (temporarily) when another error is corrected.
- The symptom may actually be caused by non errors (e.g., round-off inaccuracies).

Debugging Strategies

- In general, **three** debugging strategies have been proposed
- 1.brute force,
- 2.backtracking, and
- 3.cause elimination.
- Each of these strategies can be conducted manually, but modern debugging tools can make the process much more effective.

Debugging tactics.

- The **brute force** category of debugging is probably the most common and least efficient method for isolating the cause of a software error. You apply brute force debugging methods when **all else fails**.
- **Backtracking** is a fairly common debugging approach that can be used successfully in small programs. Beginning at the site where a symptom has been uncovered, the source code is traced backward (manually) until the cause is found. Unfortunately, as the number of source lines increases, the number of potential backward paths may become unmanageably large.

- The third approach to debugging is **cause elimination**. It is manifested by induction or deduction and introduces the concept of binary partitioning.

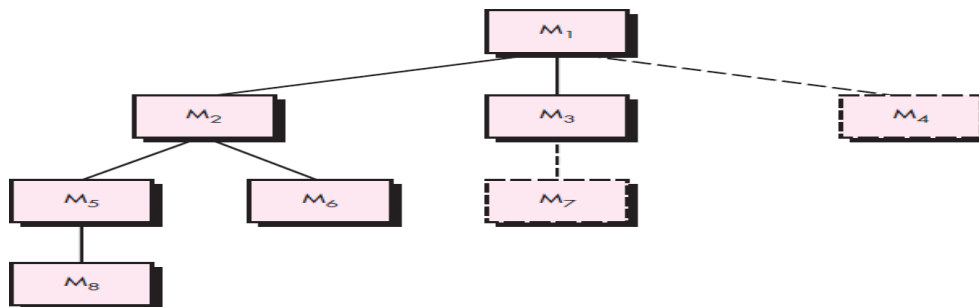
- **Correcting the Error**

Once a bug has been found, it must be corrected. But, as we have already noted, the correction of a bug can introduce other errors and therefore do more harm than good.

7. Discuss about Integration Testing

Integration Testing

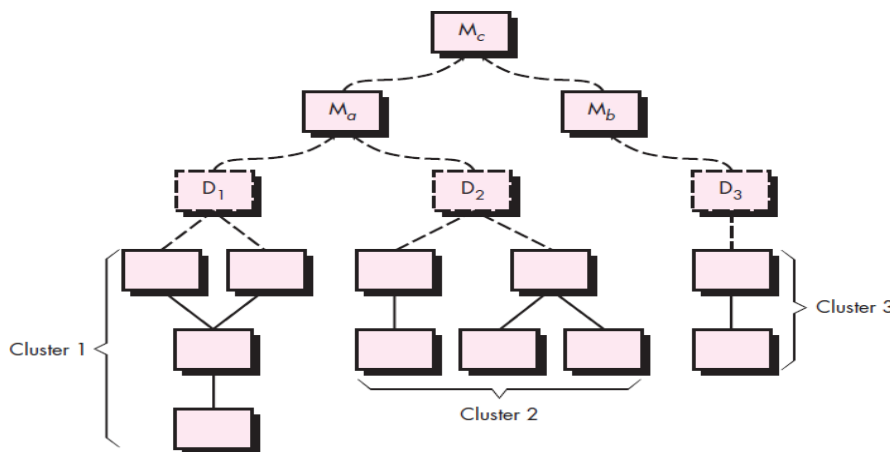
- Integration testing is a systematic technique for constructing the software architecture while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit-tested components and build a program structure that has been dictated by design.
- There is often a tendency to attempt non incremental integration; that is, to construct the program using a “**big bang**” approach. All components are combined in advance. The entire program is tested as a **whole**.
- **Top-down integration.** *Top-down integration testing* is an incremental approach to construction of the software architecture. Modules are integrated by moving downward through the control hierarchy, beginning with the main control module (main program). Modules subordinate to the main control module are incorporated into the structure in either a **depth-first or breadth-first** manner.



- **depth-first integration** integrates all components on a major control path of the program structure. For example, selecting the left-hand path, components M1, M2, M5 would be integrated first. Next, M8 or M6 would be integrated. Then, the central and right-hand control paths are built.
- **Breadth-first integration** incorporates all components directly subordinate at each level, moving across the structure horizontally. From the figure, components M2, M3, and M4 would be integrated first. The next control level, M5, M6, and so on, follows.

Bottom-up integration. *Bottom-up integration testing*, as its name implies, begins construction and testing with **atomic modules** (i.e., components at the lowest levels in the program structure). Because components are integrated from the bottom up, the functionality provided by components subordinate to a given level is always available and the need for stubs is eliminated. A bottom-up integration strategy may be implemented with the following steps:

- Low-level components are combined into clusters (sometimes called *builds*) that perform a specific software sub function.
- A *driver* (a control program for testing) is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure.



Integration follows the pattern illustrated in following figure. Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver (shown as a dashed block). Components in clusters 1 and 2 are subordinate to M_a . Drivers D_1 and D_2 are removed and the clusters are interfaced directly to M_a . Similarly, driver D_3 for cluster 3 is removed prior to integration with module M_b . Both M_a and M_b will ultimately be integrated with component M_c , and so forth.

8. Compare black box and white box testing.

Black Box testing

- In [Black-box testing](#), a tester doesn't have any information about the internal working of the software system. Black box testing is a high level of testing that focuses on the behavior of the software. It involves testing from an external or end-user perspective. Black box testing can be applied to virtually every level of software testing: unit, integration, system, and acceptance.
- In other words, we can say that black box testing is a process of checking the functionality of an application as per the customer's requirement. Mainly, there are three types of black-box

testing: **functional testing**, **Non-Functional testing**, and **Regression testing**. Its main objective is to specify the business needs or the customer's requirements.

White Box testing

- [White-box testing](#) is a testing technique which checks the internal functioning of the system. In this method, testing is based on coverage of code statements, branches, paths or conditions. White-Box testing is considered as low-level testing. It is also called glass box, transparent box, clear box or code base testing. The white-box Testing method assumes that the path of the logic in a unit or program is known.
- It is performed by Developers, and then the software will be sent to the testing team, where they perform black-box testing. The main objective of white-box testing is to test the application's infrastructure. It is done at lower levels, as it includes unit testing and integration testing.
- **KEY DIFFERENCE**
 - In Black Box, testing is done without the knowledge of the internal structure of program or application whereas in White Box, testing is done with knowledge of the internal structure of program.
 - When we compare Blackbox and Whitebox testing, Black Box test doesn't require programming knowledge whereas the White Box test requires programming knowledge.
 - Black Box testing has the main goal to test the behavior of the software whereas White Box testing has the main goal to test the internal operation of the system.
 - Comparing White box testing and Black box testing, Black Box testing is focused on external or end-user perspective whereas White Box testing is focused on code structure, conditions, paths and branches.
 - Black Box test provides low granularity reports whereas the White Box test provides high granularity reports.

Conclusion

- Black Box Testing focuses on testing the software's functionality without knowledge of its internal structure, while White Box Testing examines the internal code and logic. Black Box Testing is suited for functional testing at higher levels, conducted by testers, while White Box Testing is more suitable for unit and integration testing, performed by developers. Both have their advantages and disadvantages, and the choice depends on project requirements and desired test coverage.

9. Describe metrics for Software Quality.

METRICS FOR SOFTWARE QUALITY

- The quality of a system, application, or product is only as good as the requirements that describe the problem, the design that models the solution, the code that leads to an executable program, and the tests that exercise the software to uncover errors. You can use measurement to assess the quality of the requirements and design models, the source code, and the test cases that have been created as the software is engineered.

- **Measuring Quality**

Although there are many measures of software quality, correctness, maintainability, integrity, and usability provide useful indicators for the project team.

- **Correctness.** Correctness is the degree to which the software performs its required function. Defects (lack of correctness) are those problems reported by a user of the program after the program has been released for general use. For quality assessment purposes, defects are counted over a standard period of time, typically one year. The most common measure for correctness is defects per KLOC, where a defect is defined as a verified lack of conformance to requirements.
- **Maintainability.** Maintainability is the ease with which a program can be corrected if an error is encountered, adapted if its environment changes, or enhanced if the customer desires a change in requirements. There is no way to measure maintainability directly; therefore, we must use indirect measures. A simple time-oriented metric is mean-time-to-change (MTTC), the time it takes to analyze the change request, design an appropriate modification, implement the change, test it, and distribute the change to all users.
- **Integrity.**

This attribute measures a system's ability to withstand attacks (both accidental and intentional) to its security. To measure integrity, two additional attributes must be defined: threat and security. Threat is the probability (which can be estimated or derived from empirical evidence)

that an attack of a specific type will occur within a given time. Security is the probability (which can be estimated or derived from empirical evidence) that the attack of a specific type will be repelled

The integrity of a system can then be defined as:

$$\text{Integrity} = \sum [1 - (\text{threat} * (1 - \text{security}))]$$

- **Usability.** Usability is an attempt to quantify ease of use and can be measured in terms of the characteristics

10.Explain Size-Oriented and Function-Oriented Metrics.

Size-Oriented Metrics

- Size-oriented software metrics are derived by normalizing quality and/or productivity measures by considering the *size of the software that has been produced*. If a software organization maintains simple records, a table of size-oriented measures, such as the one shown in Figure can be created. The table lists each software development project that has been completed over the past few years and corresponding measures for that project.

FIGURE 32.2

Size-oriented
metrics

Project	LOC	Effort	\$(000)	Pp. doc.	Errors	Defects	People
alpha	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gamma	20,200	43	314	1050	256	64	6
•	•	•	•	•	•		
•	•	•	•	•	•		
•	•	•	•	•	•		

- Referring to the table entry for project *alpha* : *12,100 lines of code were developed* with 24 person-months of effort at a cost of \$168,000. It should be noted that the effort and cost recorded in the table represent all software engineering activities (analysis, design, code, and test), not just coding. Further information for project *alpha* indicates that *365 pages of documentation were developed*, *134 errors* were recorded before the software was released, and 29 defects were encountered after release to the customer within the first year of operation. Three people worked on the development of software for project alpha.
- From the rudimentary data contained in the table, a set of simple size-oriented metrics can be developed for each project:
 - Errors per KLOC (thousand lines of code)
 - Defects per KLOC
 - \$ per KLOC
 - Pages of documentation per KLOCIn addition, other interesting metrics can be computed:
 - Errors per person-month
 - KLOC per person-month
 - \$ per page of documentation

Function-Oriented Metrics

- Function-oriented software metrics use a measure of the functionality delivered by the application as a normalization value. The most widely used function-oriented metric is the

function point (FP). Computation of the function point is based on characteristics of the software's information domain and complexity.

The function point, like the LOC measure, is controversial. **Proponents claim** that FP is programming language-independent, making it ideal for applications using conventional and nonprocedural languages, and that it is based on data that are more likely to be known early in the evolution of a project, making FP more attractive as an estimation approach. **Opponents claim** that the method requires some “sleight of hand” in that computation is based on subjective rather than objective data, that counts of the information domain (and other dimensions) can be difficult to collect after the fact, and that FP has no direct physical meaning—it's just a number.

SE UNIT-5 SOLUTIONS

SHORT ANSWER SOLUTIONS

1.What is Reactive Strategy?

A reactive strategy monitors the project for likely risks. Resources are set aside to deal with them, should they become actual problems. More commonly, the software team does nothing about risks until something goes wrong. Then, the team flies into action in an attempt to correct the problem rapidly. This is often called a *fire-fighting mode*.

2.What is Proactive Strategy?

A proactive strategy begins long before technical work is initiated. Potential risks are identified, their probability and impact are assessed, and they are ranked by importance. Then, the software team establishes a plan for managing risk. The primary objective is to avoid risk, but because not all risks can be avoided, the team works to develop a contingency plan that will enable it to respond in a controlled and effective manner.

3. What are Business Risks?

Business risks threaten the viability of the software to be built and often jeopardize the project or the product. Candidates for the top five business risks are

- a. Market risk
- b. Strategic risk
- c. sales risk
- d. management risk
- e. budget risks

4. What is RMMM.

RMMM stands for Risk Mitigation, Monitoring and Management

RiskMitigation

It is an activity used to avoid problems (Risk Avoidance).

RiskMonitoring

It is an activity used for project tracking.

Risk Management

Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality.

5.What are Project Risks?

Project risks threaten the project plan. That is, if project risks become real, it is likely that the project schedule will slip and that costs will increase. Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, stakeholder, and requirements problems and their impact on a software project

6. What is software reliability?

Software reliability is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time".

7. Define Quality.

- *Quality of design refers to the characteristics that designers specify for an item.*
- *Quality of conformance is the degree to which the design specifications are followed during manufacturing.*

8. What are Internal and External Failure Costs?

Internal failure costs are incurred when we detect a defect in our product prior to shipment. Internal failure costs include

- rework repair
- failure mode analysis

External failure costs are associated with defects found after the product has been shipped to the customer. Examples of external failure costs are

- complaint resolution
- product return and replacement help line support
- warranty work

9. What is Software Review?

Software reviews are a "filter" for the software engineering process. That is, reviews are applied at various points during software development and serve to uncover errors and defects that can then be removed. Software reviews "purify" the software engineering activities that we have called *analysis*, *design*, and *coding*.

10. List the different categories of software Risks.

- a. Project risks
- b. Technical risks
- c. Business risks
- d. Known risks
- e. Predictable risks
- f. Unpredictable risks

SE UNIT-5 LONG ANSWER SOLUTIONS

1. Discuss about Software Risks.

SOFTWARE RISKS

There is general agreement that risk always involves two characteristics:

uncertainty —the risk may or may not happen; that is, there are no 100 percent probable risks

loss —if the risk becomes a reality, unwanted consequences or losses will occur.

- When risks are analyzed, it is important to quantify the level of uncertainty and the degree of loss associated with each risk. To accomplish this, different categories of risks are considered

Project risks threaten the project plan. That is, if project risks become real, it is likely that the project schedule will slip and that costs will increase. Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, stakeholder, and requirements problems and their impact on a software project.

Technical risks threaten the quality and timeliness of the software to be produced. If a technical risk becomes a reality, implementation may become difficult or impossible. Technical risks identify potential design, implementation, interface, verification, and maintenance problems.

Business risks threaten the viability of the software to be built and often jeopardize the project or the product. Candidates for the top five business risks are

- (1) building an excellent product or system that no one really wants (market risk),
- (2) building a product that no longer fits into the overall business strategy for the company (strategic risk),
- (3) building a product that the sales force doesn't understand how to sell (sales risk),
- (4) losing the support of senior management due to a change in focus or a change in people (management risk), and
- (5) losing budgetary or personnel commitment (budget risks).

Known risks are those that can be uncovered after careful evaluation of the project plan, the business and technical environment in which the project is being developed, and other reliable information sources (e.g., unrealistic delivery date, lack of documented requirements or software scope, poor development environment).

Predictable risks are extrapolated from past project experience (e.g., staff turnover, poor communication with the customer, dilution of staff effort as ongoing maintenance requests are serviced).

Unpredictable risks are the joker in the deck. They can and do occur, but they are extremely difficult to identify in advance.

2. Compare Reactive Vs Proactive Risk Strategies.

REACTIVE VERSUS PROACTIVE RISK STRATEGIES

- At best, a **reactive strategy** monitors the project for likely risks. Resources are set aside to deal with them, should they become actual problems. More commonly, the software team does nothing about risks until something goes wrong. Then, the team flies into action in an attempt to correct the problem rapidly. This is often called a *fire-fighting mode*.

- A considerably more intelligent strategy for risk management is to be proactive. **A *proactive strategy* begins long before technical work is initiated.** Potential risks are identified, their probability and impact are assessed, and they are ranked by importance. Then, the software team establishes a plan for managing risk. The primary objective is to avoid risk, but because not all risks can be avoided, the team works to develop a contingency plan that will enable it to respond in a controlled and effective manner.

3. Discuss about risk identification.

Risk identification is a systematic attempt to specify threats to the project plan (estimates, schedule, resource loading, etc.). By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary.

There are two distinct types of risks.

Generic risks and product-specific risks.

Generic risks are a potential threat to every software project.

Product-specific risks can be identified only by those with a clear understanding of the technology, the people, and the environment that is specific to the project that is to be built.

One method for identifying risks is to create a *risk item checklist*. The checklist can be used for risk identification and focuses on some subset of known and predictable risks in the following generic subcategories:

- **Product size** —Risks associated with the overall size of the software to be built or modified.
- **Business impact** —Risks associated with constraints imposed by management or the marketplace.
- **Stakeholder characteristics** —Risks associated with the sophistication of the stakeholders and the developer's ability to communicate with stakeholders in a timely manner.
- **Process definition** —Risks associated with the degree to which the software process has been defined and is followed by the development organization.
- **Development environment** —Risks associated with the availability and quality of the tools to be used to build the product.
- **Technology to be built** —Risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- **Staff size and experience** —Risks associated with the overall technical and project experience of the software engineers who will do the work.

4.Explain six sigma for software engineering.

Six Sigma for software Engineering:

Six Sigma is the most widely used strategy for statistical quality assurance in industry today.

The term —six sigma|| is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high quality standard. The Six Sigma methodology defines three core steps:

Define customer requirements and deliverables and project goals via well-defined methods of customer communication

Measure the existing process and its output to determine current quality performance (collect defect metrics)

Analyze defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps.

Improve the process by eliminating the root causes of defects.

Control the process to ensure that future work does not reintroduce the causes of defects These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

If any organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

Design the process to avoid the root causes of defects and to meet customer requirements

Verify that the process model will, in fact, avoid defects and meet customer requirements. This variation is sometimes called the DMADV (define, measure, analyze, design and verify) method.

5. Explain RMMM.

Risk Mitigation

- It is an activity used to avoid problems (Risk Avoidance). Steps for mitigating the risks as follows.
- Finding out the risk.
- Removing causes that are the reason for risk creation.
- Controlling the corresponding documents from time to time.
- Conducting timely reviews to speed up the work.

Risk Monitoring

- It is an activity used for project tracking. It has the following primary objectives as follows.
To check if predicted risks occur or not.
- To ensure proper application of risk aversion steps defined for risk.
- To collect data for future risk analysis.
- To allocate what problems are caused by which risks throughout the project.

Risk Management and planning

It assumes that the mitigation activity failed and the risk is a reality. This task is done by Project manager when risk becomes reality and causes severe problems. If the project manager effectively uses project mitigation to remove risks successfully then it is easier to manage the risks. This shows that the response that will be taken for each risk by a manager. The main objective of the risk management plan is the risk register. This risk register describes and focuses on the predicted threats to a software project.

Example:

- Let us understand RMMM with the help of an example of high staff turnover.

Risk Mitigation:

- To mitigate this risk, project management must develop a strategy for reducing turnover. The possible steps to be taken are:
- Meet the current staff to determine causes for turnover (e.g., poor working conditions, low pay, competitive job market).
- Mitigate those causes that are under our control before the project starts.
- Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
- Organize project teams so that information about each development activity is widely dispersed.
- Define documentation standards and establish mechanisms to ensure that documents are developed in a timely manner.
- Assign a backup staff member for every critical technologist.

Risk Monitoring:

- As the project proceeds, risk monitoring activities commence. The project manager monitors factors that may provide an indication of whether the risk is becoming more or less likely. In the case of high staff turnover, the following factors can be monitored:
- General attitude of team members based on project pressures.
- Interpersonal relationships among team members.

- Potential problems with compensation and benefits.
- The availability of jobs within the company and outside it.

Risk Management:

- Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality. Continuing the example, the project is well underway, and a number of people announce that they will be leaving. If the mitigation strategy has been followed, backup is available, information is documented, and knowledge has been dispersed across the team. In addition, the project manager may temporarily refocus resources (and readjust the project schedule) to those functions that are fully staffed, enabling newcomers who must be added to the team to “get up to the speed”.
- **Drawbacks of RMMM:**
 - It incurs additional project costs.
 - It takes additional time.
 - For larger projects, implementing an RMMM may itself turn out to be another tedious project.
 - RMMM does not guarantee a risk-free project, infact, risks may also come up after the project is delivered.

6.Explain Risk Projection.

RISK PROJECTION

- *Risk projection, also called risk estimation, attempts to rate each risk in two ways—the likelihood or probability that the risk is real and the consequences of the problems associated with the risk, should it occur.*

The project planner, along with other managers and technical staff, performs four risk projection activities:

- establish a scale that reflects the perceived likelihood of a risk,
- delineate the consequences of the risk,
- estimate the impact of the risk on the project and the product, and
- Assess the overall accuracy of the risk projection so that there will be no misunderstandings.

Figure 35.1Impact
assessment

Source: [Boe89].

Components		Performance	Support	Cost	Schedule
Category					
Catastrophic	1	Failure to meet the requirement would result in mission failure		Failure results in increased costs and schedule delays with expected values in excess of \$500K	
	2	Significant degradation to nonachievement of technical performance	Nonresponsive or unsupportable software	Significant financial shortages, budget overrun likely	Unachievable IOC
Critical	1	Failure to meet the requirement would degrade system performance to a point where mission success is questionable		Failure results in operational delays and/or increased costs with expected value of \$100K to \$500K	
	2	Some reduction in technical performance	Minor delays in software modifications	Some shortage of financial resources, possible overruns	Possible slippage in IOC
Marginal	1	Failure to meet the requirement would result in degradation of secondary mission		Costs, impacts, and/or recoverable schedule slips with expected value of \$1K to \$100K	
	2	Minimal to small reduction in technical performance	Responsive software support	Sufficient financial resources	Realistic, achievable schedule
Negligible	1	Failure to meet the requirement would create inconvenience or nonoperational impact		Error results in minor cost and/or schedule impact with expected value of less than \$1K	
	2	No reduction in technical performance	Easily supportable software	Possible budget underrun	Early achievable IOC

Note: (1) The potential consequence of undetected software errors or faults.
 (2) The potential consequence if the desired outcome is not achieved.

- **Developing a Risk Table**
- A risk table provides you with a simple technique for risk projection. A sample risk table is illustrated in Figure

Figure 35.2Sample risk
table prior to
sorting

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	
Σ				
Σ				
Σ				

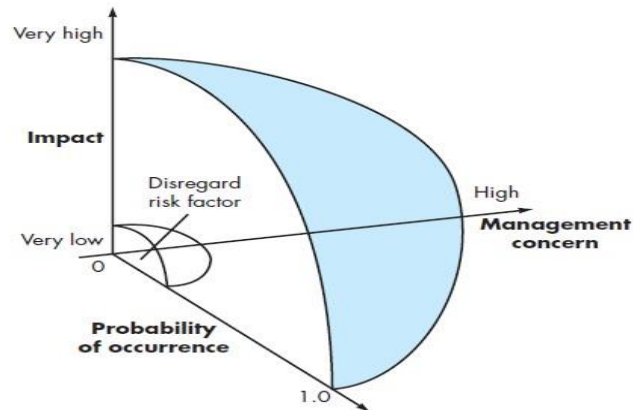
Impact values:
 1—catastrophic
 2—critical
 3—marginal
 4—negligible

- You begin by listing all risks (no matter how remote) in the first column of the table. This can be accomplished with the help of the risk item checklists. Each risk is categorized in the second

column (e.g., PS implies a project size risk, BU implies a business risk). The probability of occurrence of each risk is entered in the next column of the table. The probability value for each risk can be estimated by team members individually.

- Next, the impact of each risk is assessed. You can study the resultant sorted table and define a cutoff line. The *cutoff line* (drawn horizontally at some point in the table) implies that only risks that lie above the line will be given further attention. Risks that fall below the line are reevaluated to accomplish second-order prioritization.

FIGURE 35.3
Risk and management concern



- All risks that lie above the cutoff line should be managed. The column labeled RMMM contains a pointer into a *risk mitigation, monitoring, and management plan or, alternatively, a collection of risk information sheets developed for all risks that lie above the cutoff.*

7. Explain about Formal Technical Reviews.

FORMAL TECHNICAL REVIEWS

- A formal technical review is a software quality assurance activity performed by software engineers (and others). The objectives of the FTR are
- to uncover errors in function, logic, or implementation for any representation of the software;
- to verify that the software under review meets its requirements;
- to ensure that the software has been represented according to predefined standards;
- to achieve software that is developed in a uniform manner; and
- to make projects more manageable.

The Review Meeting

Every review meeting should abide by the following constraints:

Between three and five people (typically) should be involved in the review.

- Advance preparation should occur but should require no more than two hours of work for each person.
- The duration of the review meeting should be less than two hours.
- The focus of the FTR is on a work product.

- The individual who has developed the work product—the *producer*—informs the project leader that the work product is complete and that a review is required.
- The project leader contacts a *review leader*, who evaluates the product for readiness, generates copies of product materials, and distributes them to two or three reviewers for advance preparation.
- Each reviewer is expected to spend between one and two hours reviewing the product, making notes, and otherwise becoming familiar with the work.
- The review meeting is attended by the review leader, all reviewers, and the producer. One of the reviewers takes on the role of the *recorder*; that is, the individual who records (in writing) all important issues raised during the review.
- At the end of the review, all attendees of the FTR must decide whether to accept the product without further modification,
- reject the product due to severe errors (once corrected, another review must be performed), or accept the product provisionally.
- The decision made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team's findings.

At the end of the review meeting and a review issues list is produced. In addition, a formal technical review summary report is completed. A **review summary report** answers three questions:

- What was reviewed?
- Who reviewed it?
- What were the findings and conclusions?

8.Explain the concept of Six-Sigma for Software Engineering.

Six Sigma for software Engineering:

Six Sigma is the most widely used strategy for statistical quality assurance in industry today.

The term —six sigma|| is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high quality standard. The Six Sigma methodology defines three core steps:

- **Define** customer requirements and deliverables and project goals via well-defined methods of customer communication
- **Measure** the existing process and its output to determine current quality performance (collect defect metrics)
- **Analyze** defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps.

- **Improve** the process by eliminating the root causes of defects.
- **Control** the process to ensure that future work does not reintroduce the causes of defects These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

If any organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

- **Design** the process to
 - avoid the root causes of defects and
 - to meet customer requirements

Verify that the process model will, in fact, avoid defects and meet customer requirements. This variation is sometimes called the DMADV (define, measure, analyze, design and verify) method.

9. Discuss ISO 9000 Quality Standards.

THE ISO 9000 QUALITY STANDARDS

- A *quality assurance system* may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management
- ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered.
- ISO 9001:2000 is the quality assurance standard that applies to software engineering. The standard contains 20 requirements that must be present for an effective quality assurance system. Because the ISO 9001:2000 standard is applicable to all engineering disciplines, a special set of ISO guidelines have been developed to help interpret the standard for use in the software process.

The requirements delineated by ISO 9001 address topics such as

- management responsibility,
- quality system, contract review,
- design control,
- document and data control,
- product identification and traceability,
- process control,
- inspection and testing,
- corrective and preventive action,
- control of quality records,
- internal quality audits,
- training,
- servicing and
- statistical techniques.

In order for a software organization to become registered to ISO 9001, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed.

10. Explain about Software Reliability and Safety.

SOFTWARE RELIABILITY

- **Software reliability** is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time".

Measures of Reliability and Availability:

- Most hardware-related reliability models are predicated on failure due to wear rather than failure due to design defects. In hardware, failures due to physical wear (e.g., the effects of temperature, corrosion, shock) are more likely than a design-related failure. Unfortunately, the opposite is true for software. In fact, all software failures can be traced to design or implementation problems; wear does not enter into the picture.

- A simple measure of reliability is *meantime-between-failure* (MTBF), where
 $MTBF = MTTF + MTTR$
- The acronyms MTTF and MTTR are mean-time-to-failure and mean-time-to-repair, respectively.
- In addition to a reliability measure, we must develop a measure of availability. *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as
 $Availability = [MTTF / (MTTF + MTTR)] \times 100\%$
- The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software

Software Safety

Software safety is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail. If hazards can be identified early in the software engineering process, software design features can be specified that will either eliminate or control potential hazards.

For example, some of the hazards associated with a computer-based cruise control for an automobile might be

- causes uncontrolled acceleration that cannot be stopped
- does not respond to depression of brake pedal (by turning off) does not engage when switch is activated
- slowly loses or gains speed
- Once these system-level hazards are identified, analysis techniques are used to assign severity and probability of occurrence. To be effective, software must be analyzed in the context of the entire system.
- If a set of external environmental conditions are met (and only if they are met), the improper position of the mechanical device will cause a disastrous failure. Analysis techniques such as *fault tree analysis*